

Lecture 9

Red-Black Trees: Deletion

RB-Trees: Deletion

RB-Trees: Deletion

Two stages of deletion:

RB-Trees: Deletion

Two stages of deletion:

- Delete the node as it is done in a BST.

RB-Trees: Deletion

Two stages of deletion:

- Delete the node as it is done in a BST.
- Do fix-ups as **deletion** may cause a violation of few Red-blue properties.

RB-Trees: Deletion

Two stages of deletion:

- Delete the node as it is done in a BST.
- Do fix-ups as **deletion** may cause a violation of few Red-blue properties.

Let's recall **deletion** in a BST and **spot** nodes, say *y* and *x*, from where fix-ups can start.

Recall Deletion in BSTs

Recall Deletion in BSTs

Let z be the node we want to delete.

Recall Deletion in BSTs

Let z be the node we want to delete. Then, the following cases are possible:

Recall Deletion in BSTs

Let z be the node we want to delete. Then, the following cases are possible:

- **Case 1:** z has no children.

Recall Deletion in BSTs

Let z be the node we want to delete. Then, the following cases are possible:

- **Case 1:** z has no children.
- **Case 2:** z has only single child.

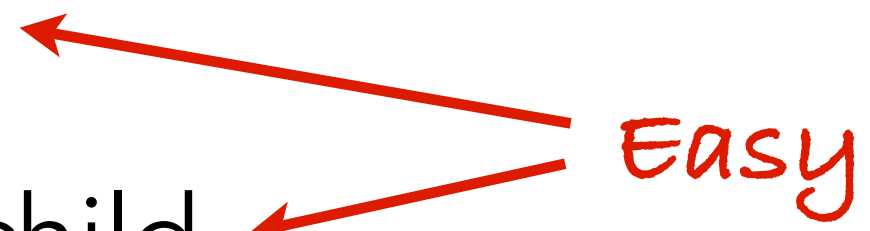
Recall Deletion in BSTs

Let z be the node we want to delete. Then, the following cases are possible:

- **Case 1:** z has no children.
- **Case 2:** z has only single child.
- **Case 3:** z has two children.

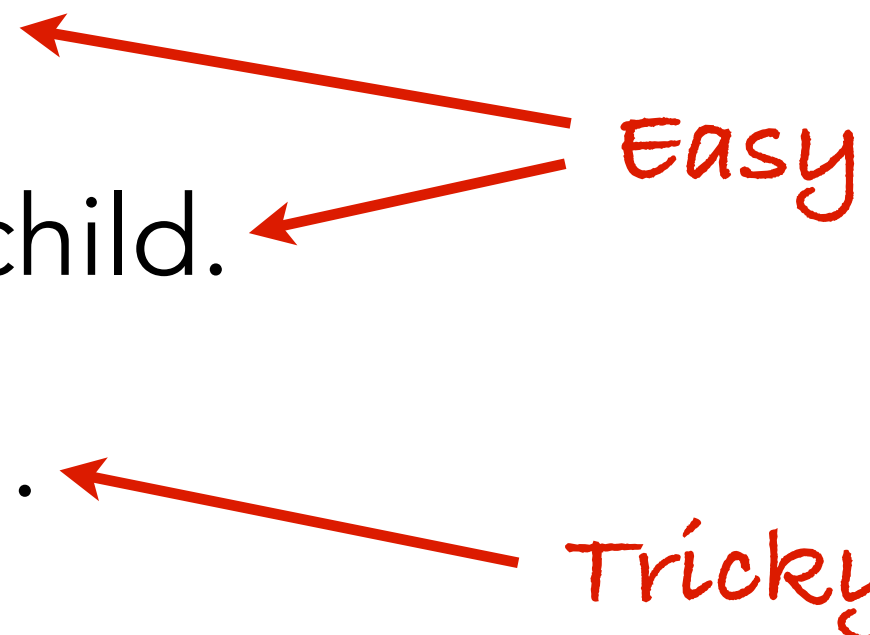
Recall Deletion in BSTs

Let z be the node we want to delete. Then, the following cases are possible:

- **Case 1:** z has no children.
 - **Case 2:** z has only single child.
 - **Case 3:** z has two children.
- 
- The word "Easy" is written in red cursive script. Two red arrows originate from its left side. One arrow points diagonally up and to the left towards the end of Case 1. The other arrow points diagonally down and to the left towards the end of Case 2.

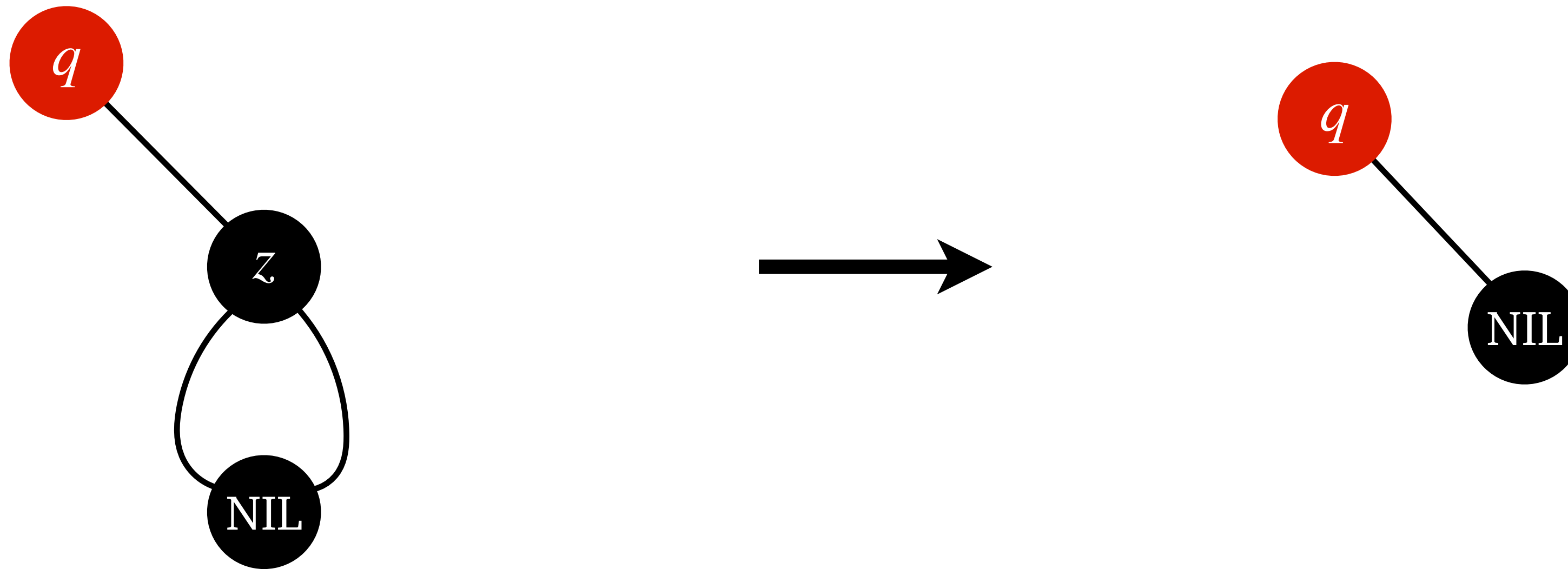
Recall Deletion in BSTs

Let z be the node we want to delete. Then, the following cases are possible:

- **Case 1:** z has no children.
 - **Case 2:** z has only single child.
 - **Case 3:** z has two children.
- 
- Easy*
- Tricky*

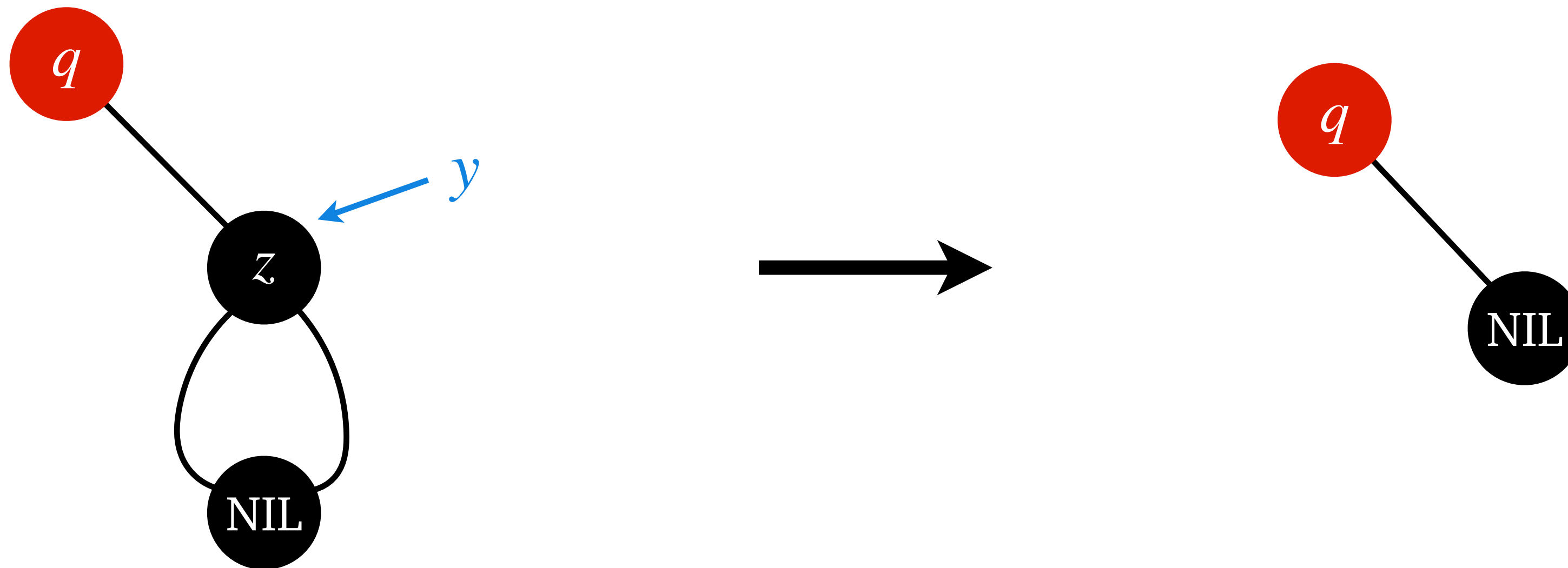
Modifying a BST: Deletion

Case 1: z has no (non-NIL) children. (WLOG assume z is a right child.)



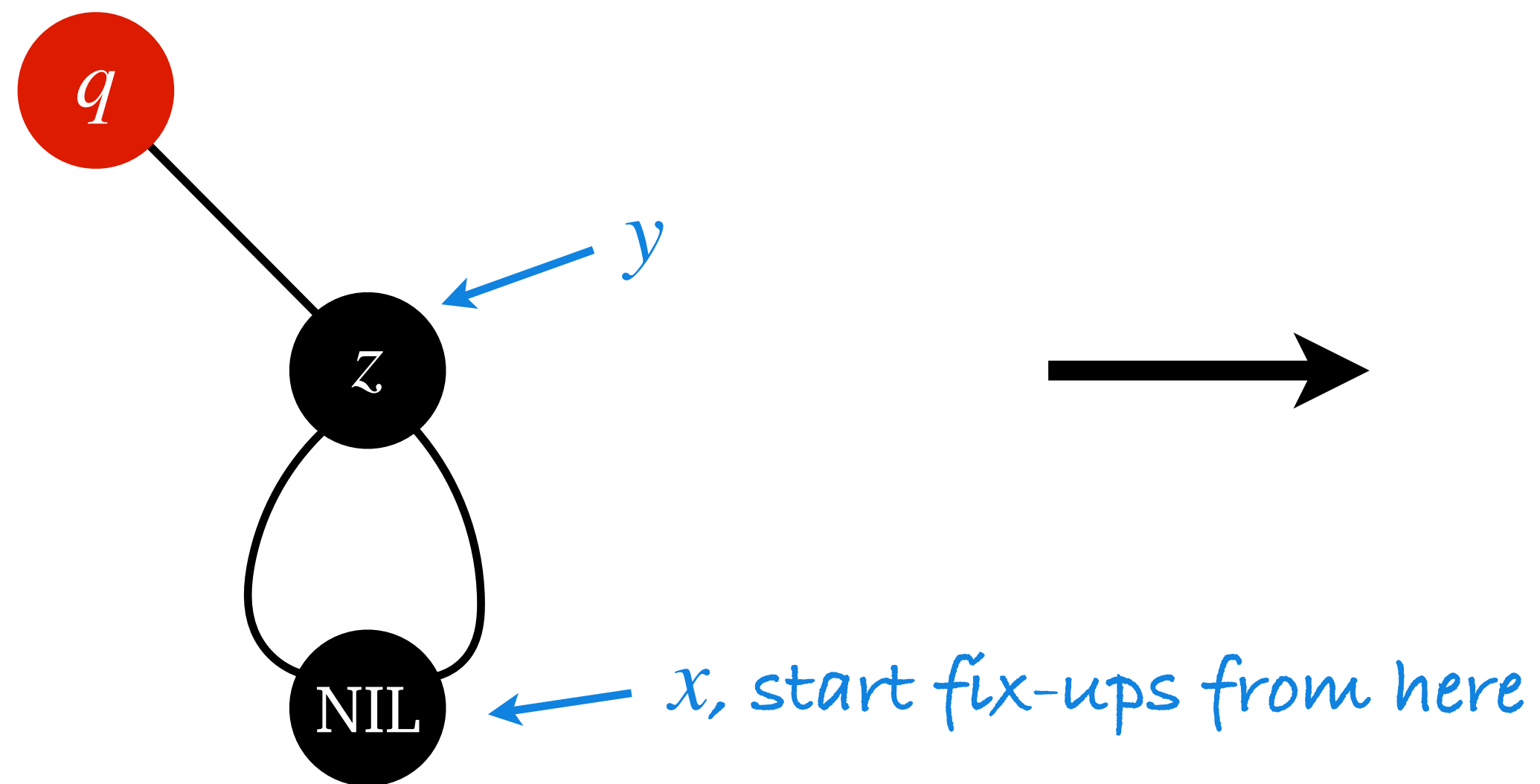
Modifying a BST: Deletion

Case 1: z has no (non-NIL) children. (WLOG assume z is a right child.)



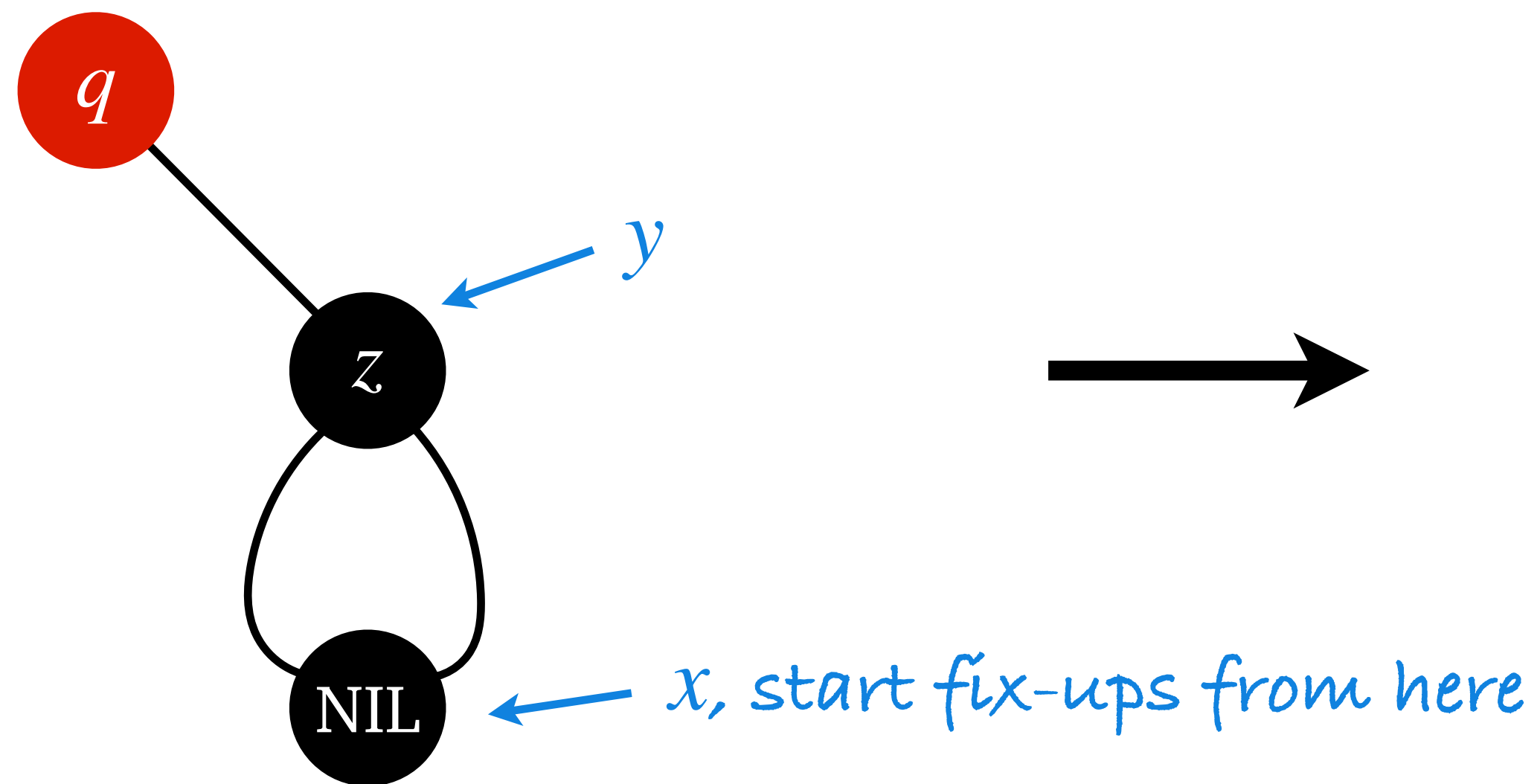
Modifying a BST: Deletion

Case 1: z has no (non-NIL) children. (WLOG assume z is a right child.)



Modifying a BST: Deletion

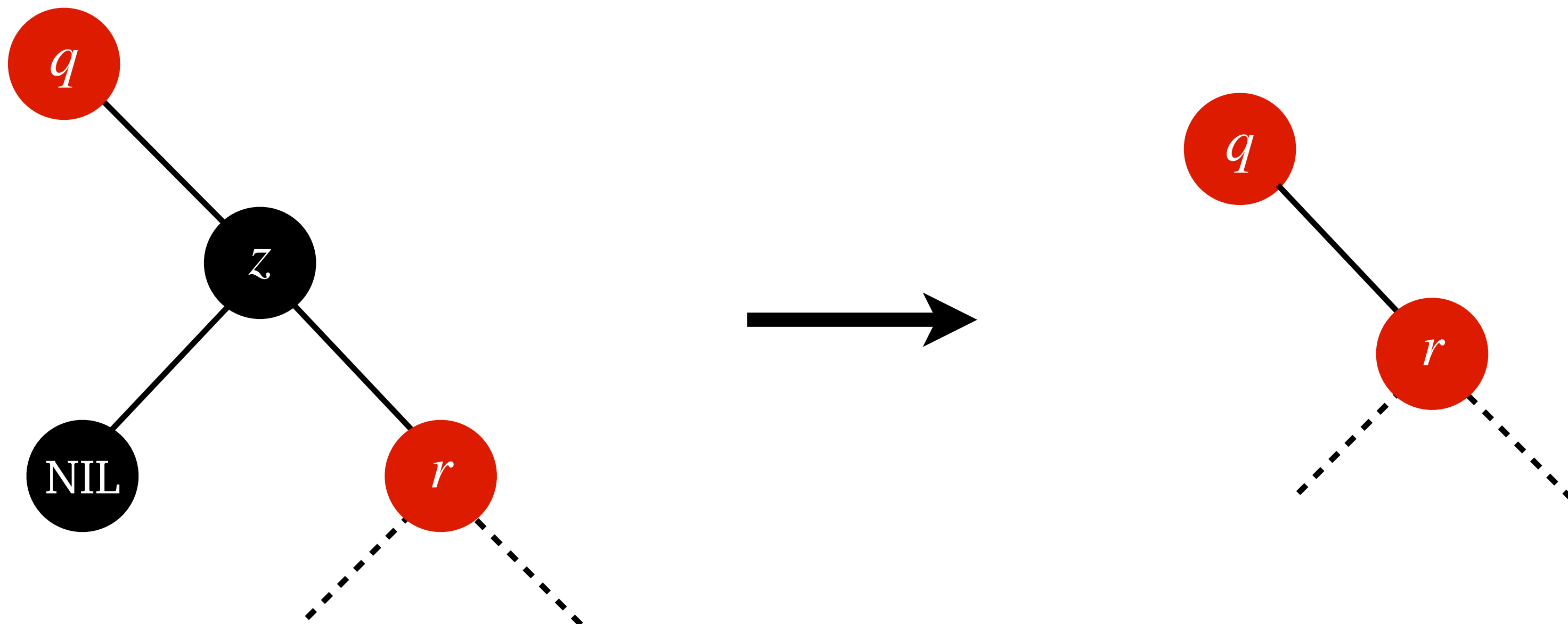
Case 1: z has no (non-NIL) children. (WLOG assume z is a right child.)



Note: In this case, y is z and x is the only child of z .

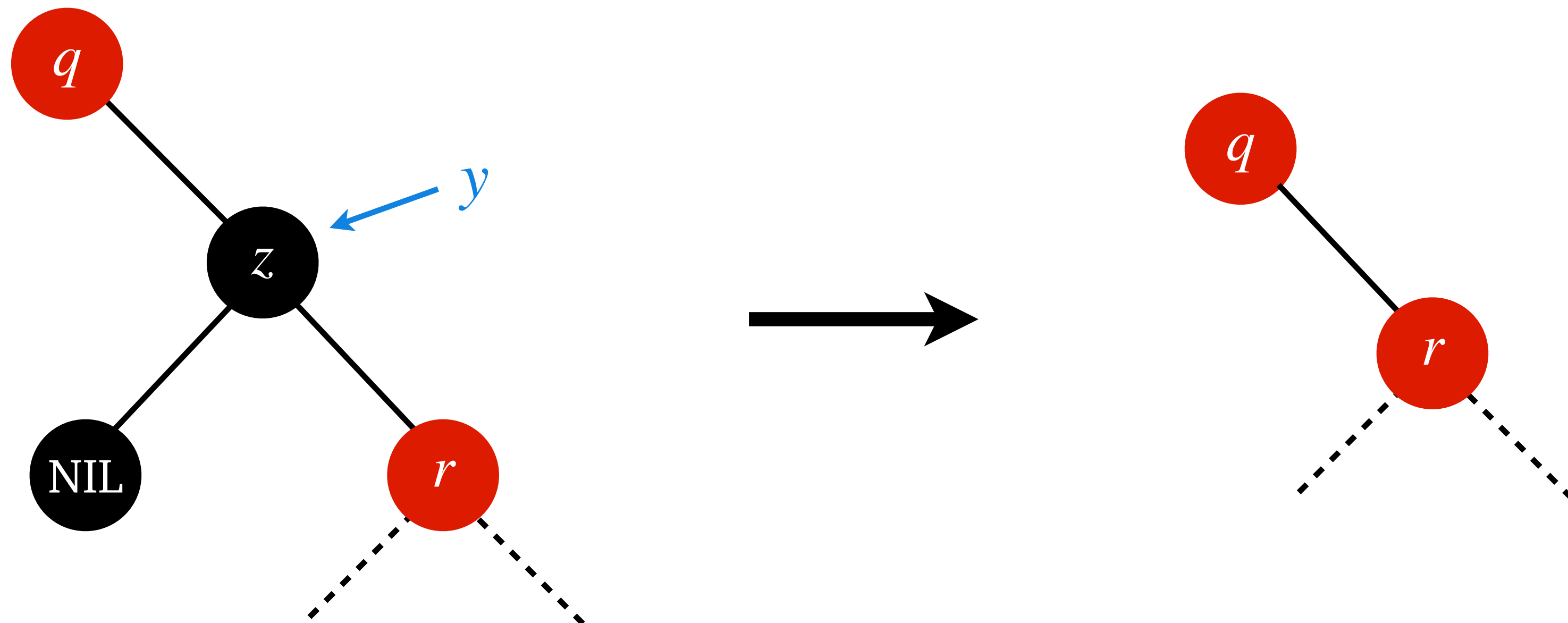
Modifying a BST: Deletion

Case 2: z has one (non-NIL) child. (WLOG assume z is a right child.)



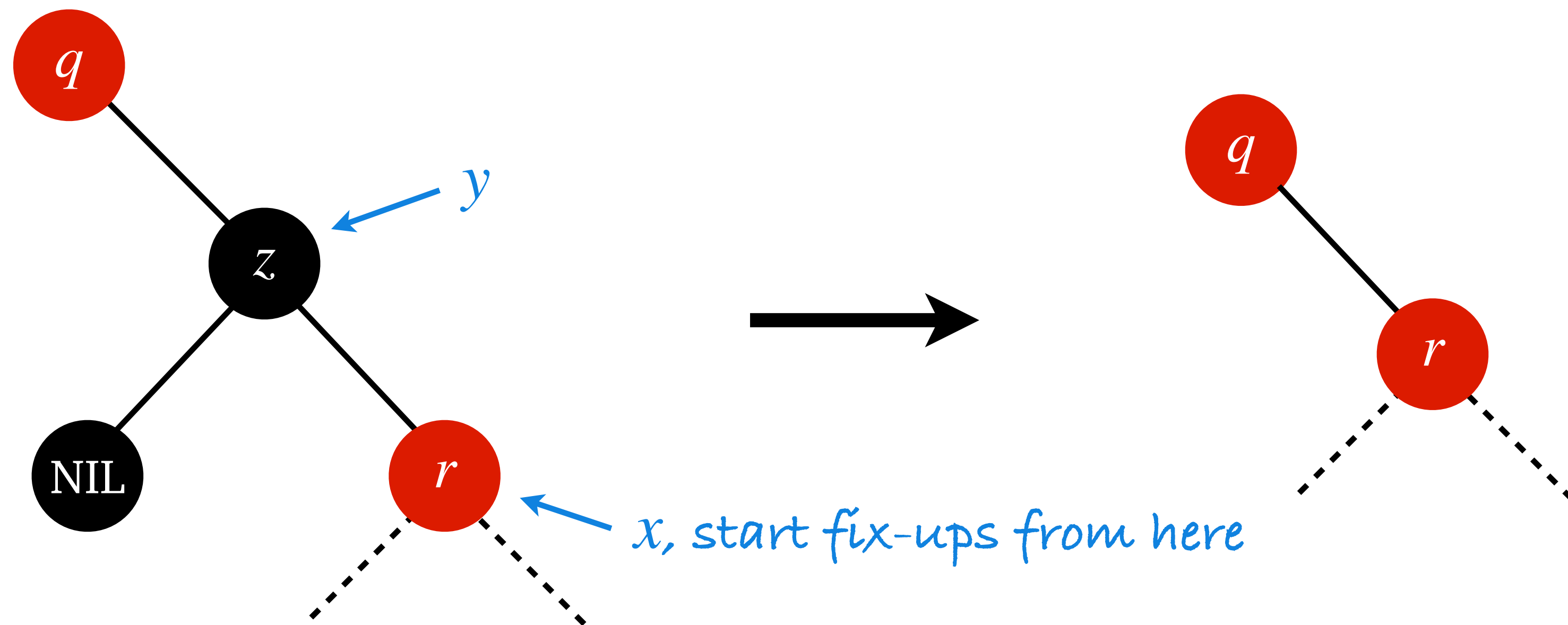
Modifying a BST: Deletion

Case 2: z has one (non-NIL) child. (WLOG assume z is a right child.)



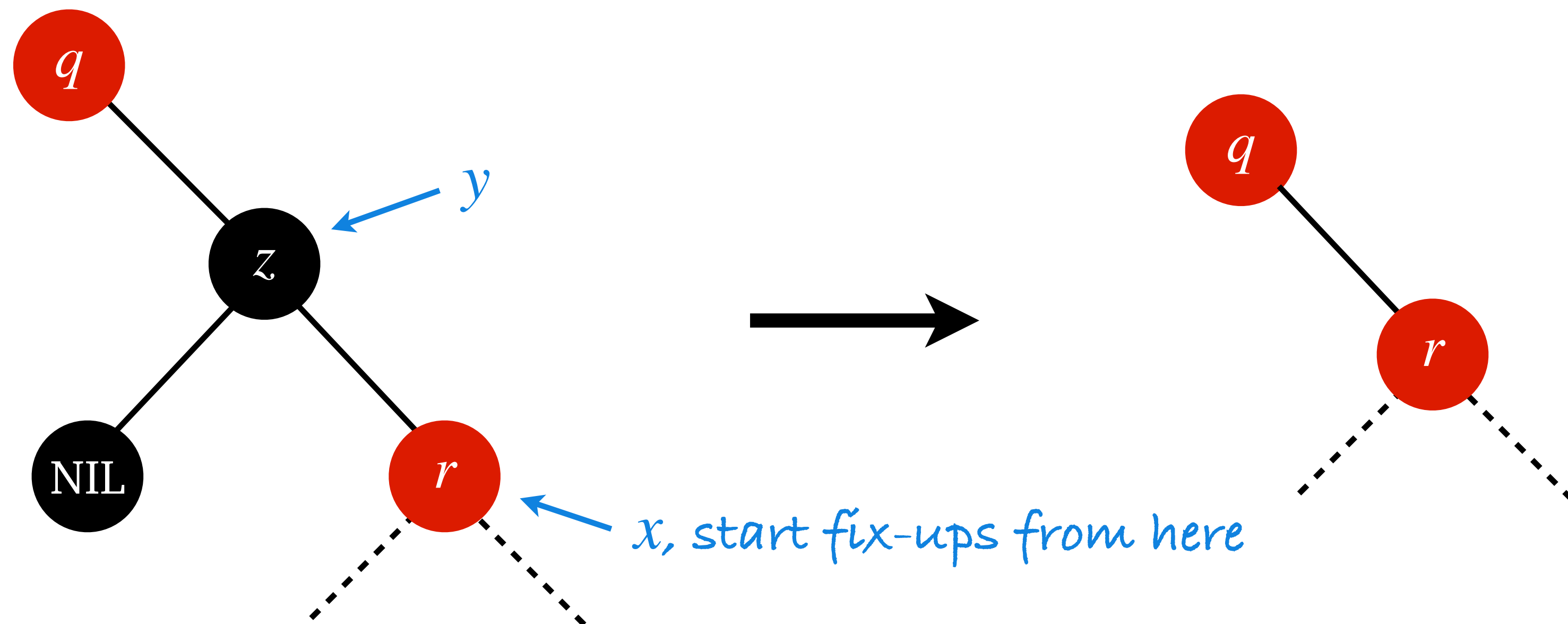
Modifying a BST: Deletion

Case 2: z has one (non-NIL) child. (WLOG assume z is a right child.)



Modifying a BST: Deletion

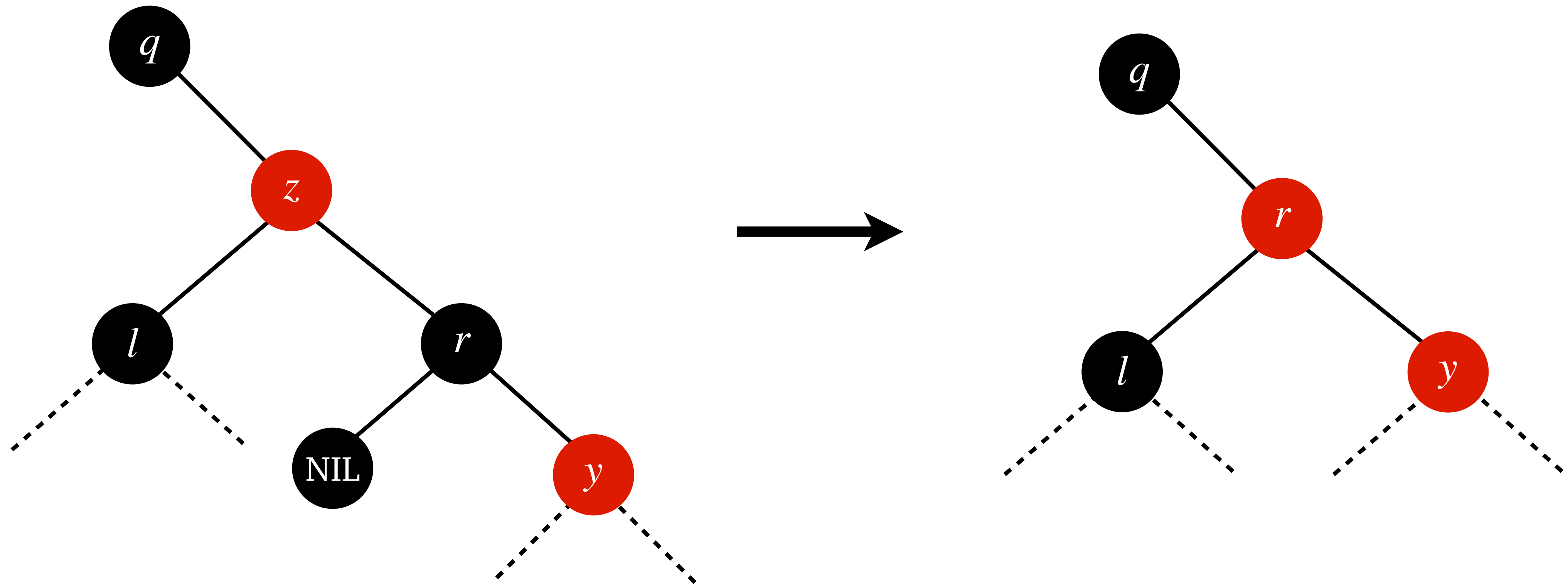
Case 2: z has one (non-NIL) child. (WLOG assume z is a right child.)



Note: In this case, y is z and x is the only child of z .

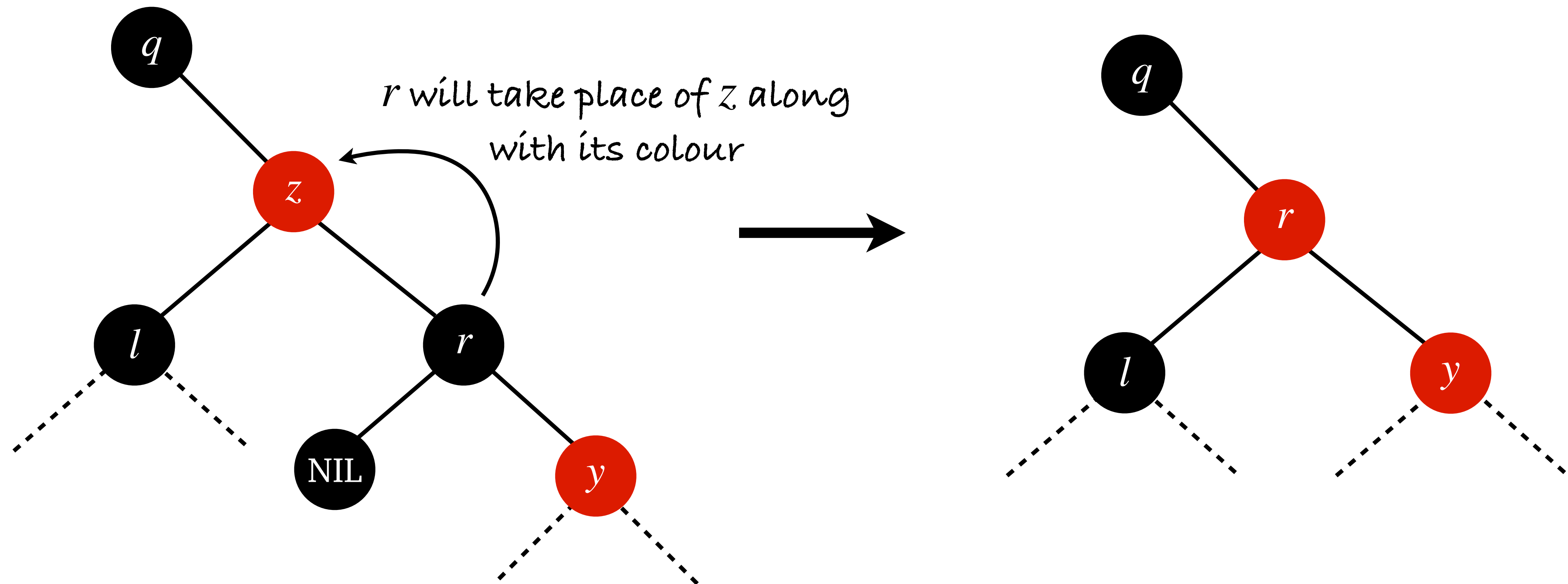
Modifying a BST: Deletion

Case 3a: z has two (non-NIL) children where its right child has no left child.



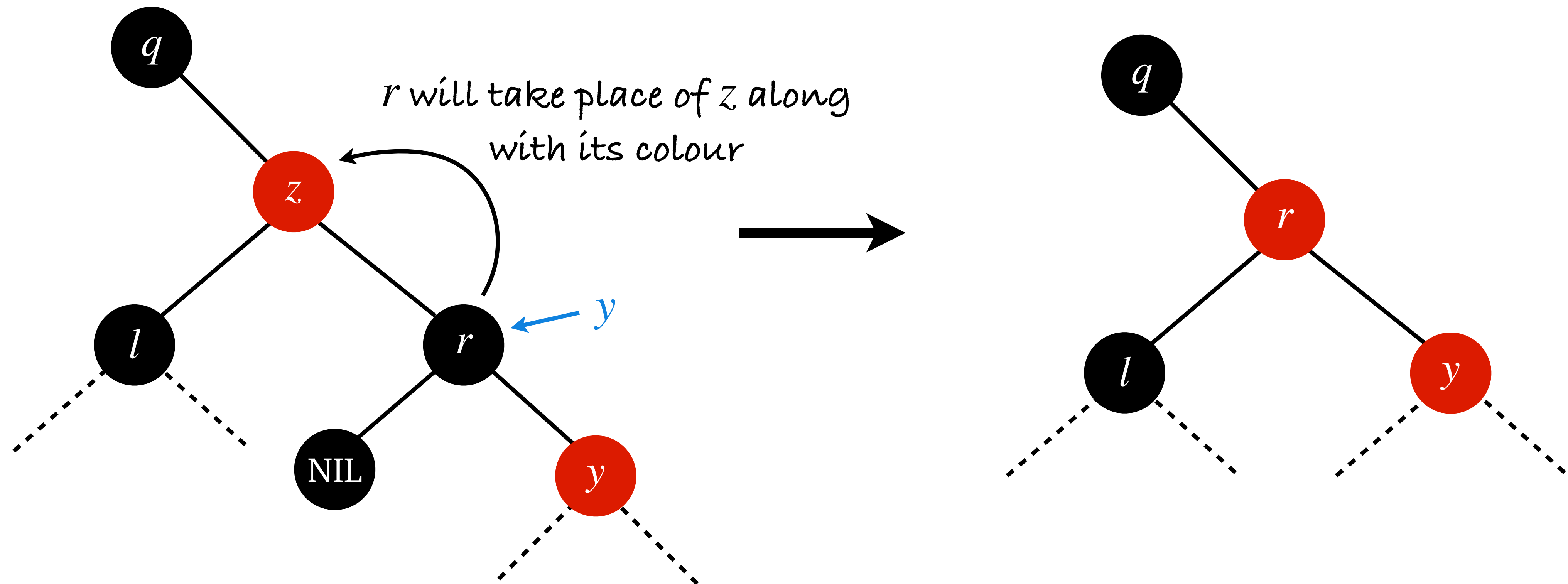
Modifying a BST: Deletion

Case 3a: z has two (non-NIL) children where its right child has no left child.



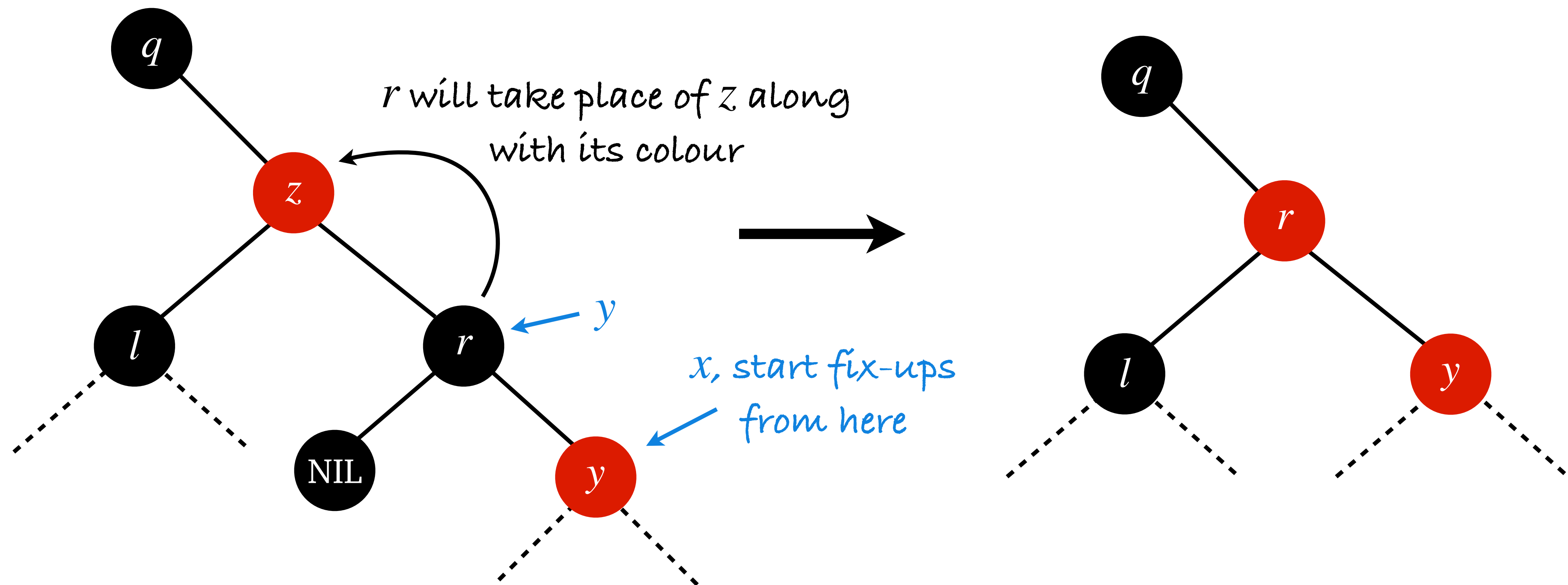
Modifying a BST: Deletion

Case 3a: z has two (non-NIL) children where its right child has no left child.



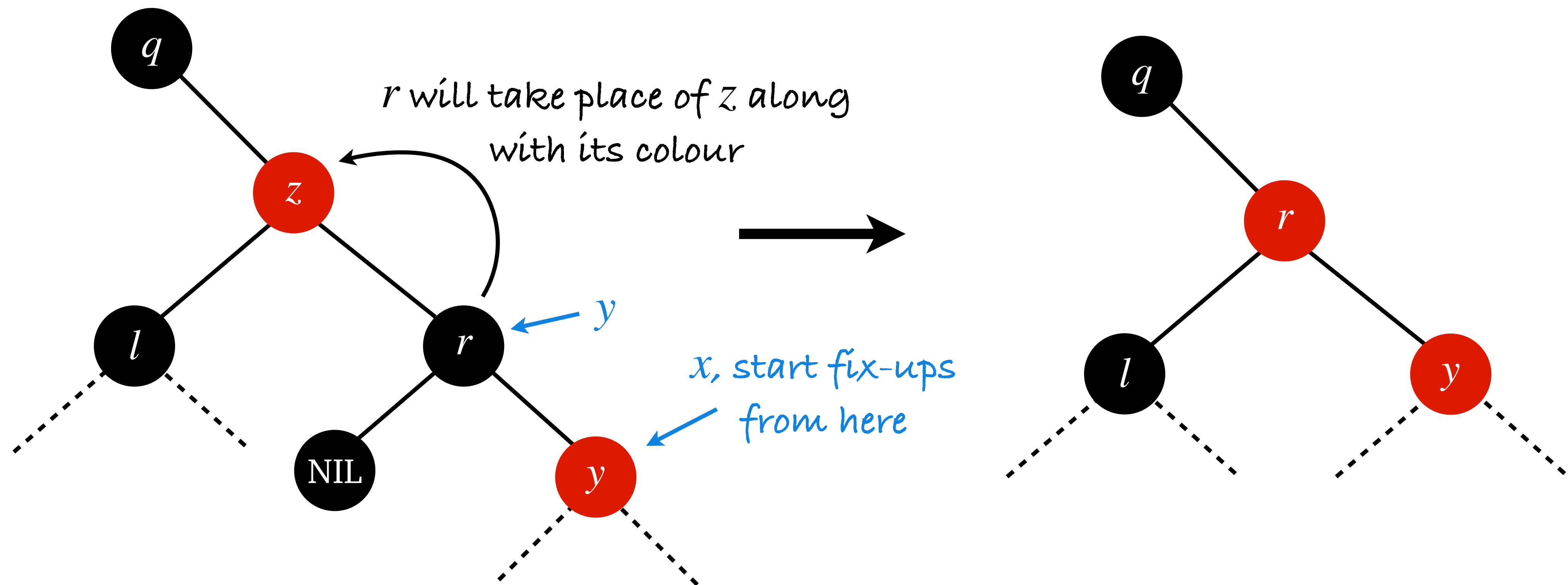
Modifying a BST: Deletion

Case 3a: z has two (non-NIL) children where its right child has no left child.



Modifying a BST: Deletion

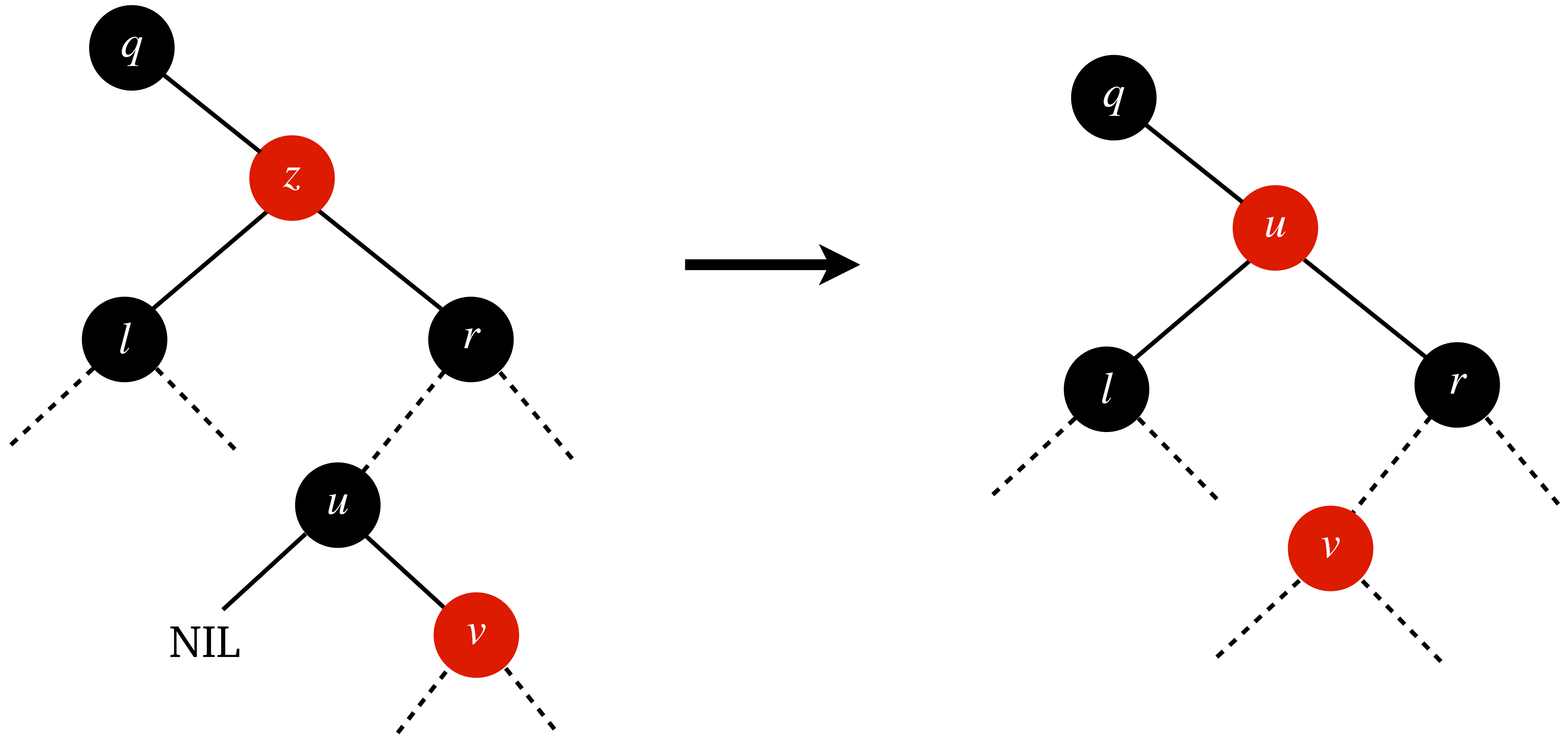
Case 3a: z has two (non-NIL) children where its right child has no left child.



Note: In this case, y is the successor of z and x is the only child of y .

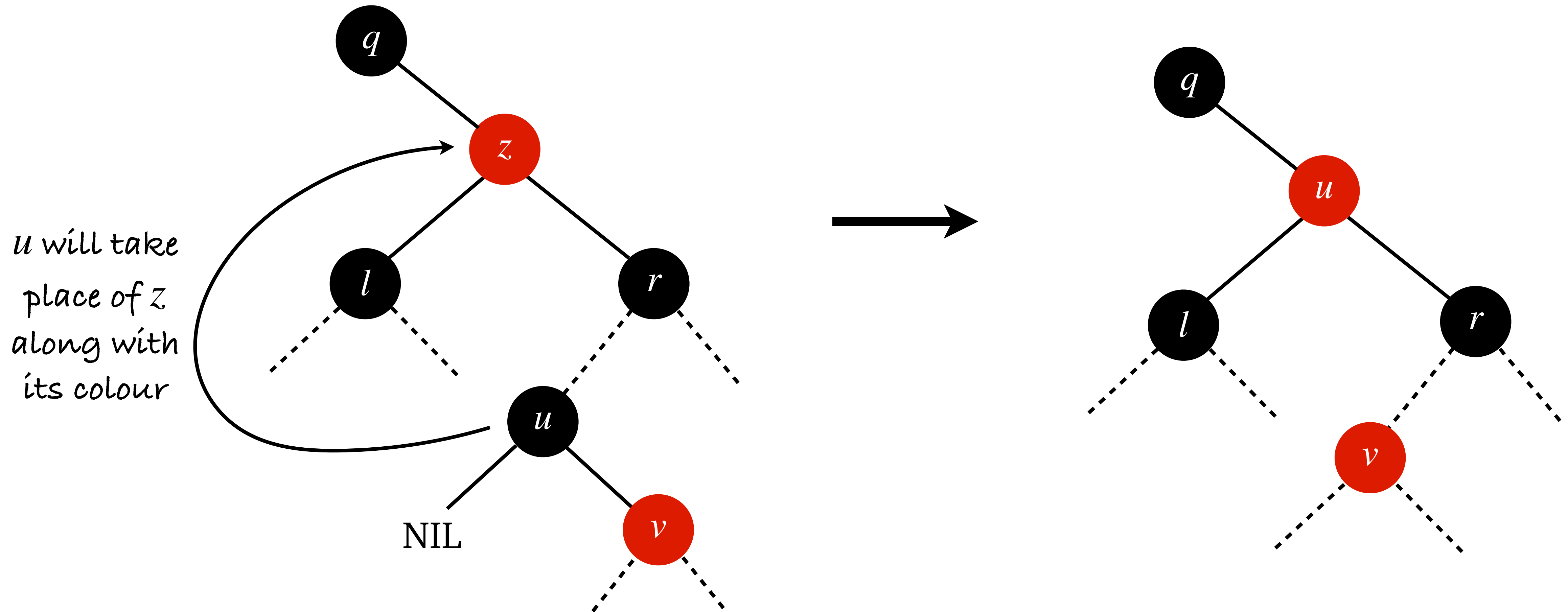
Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



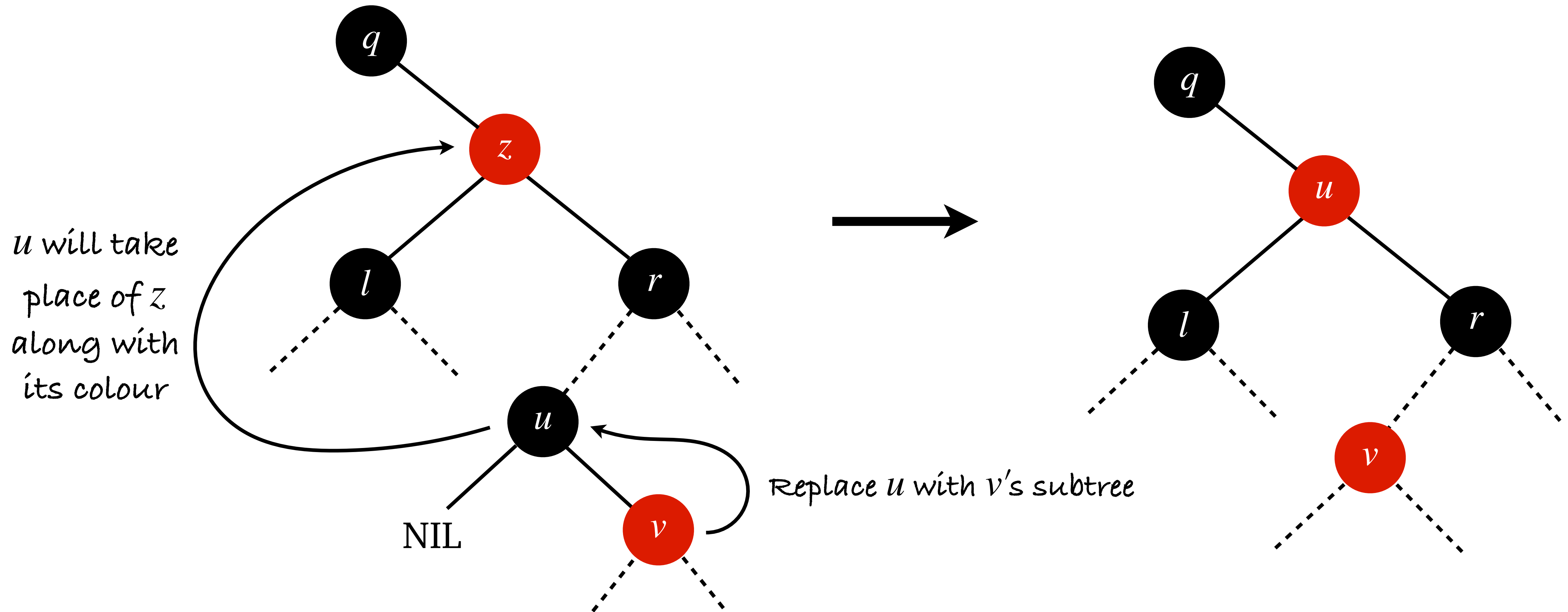
Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



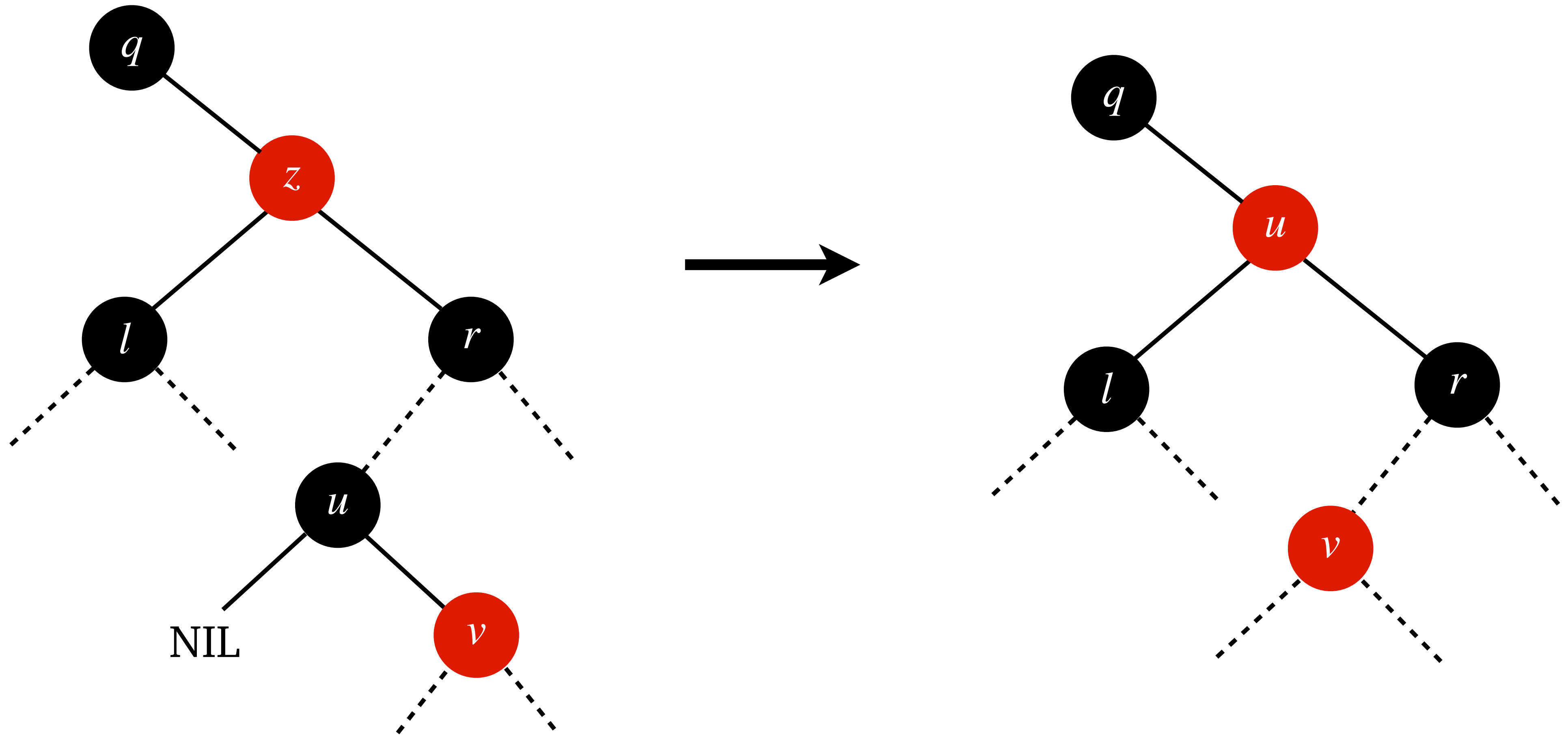
Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



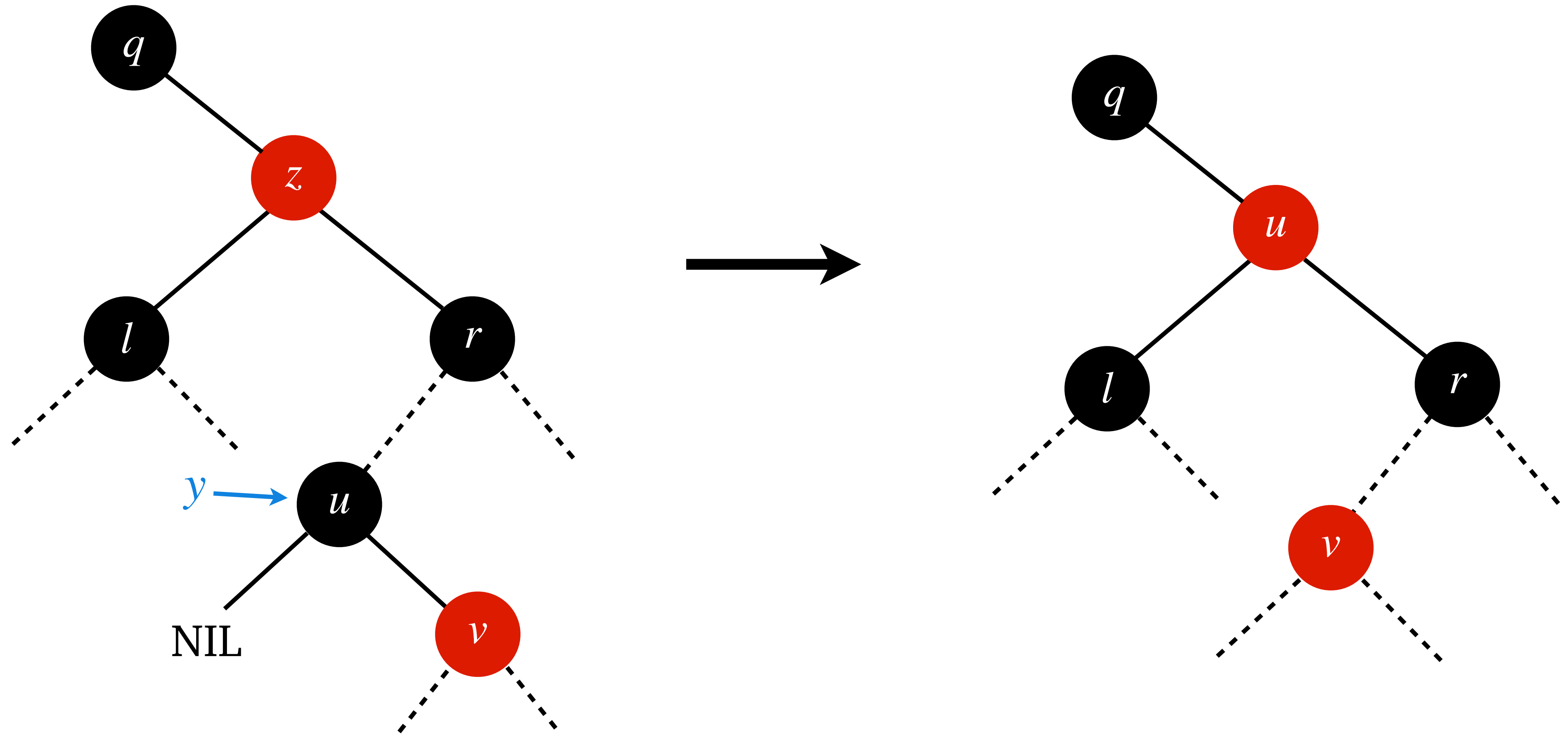
Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



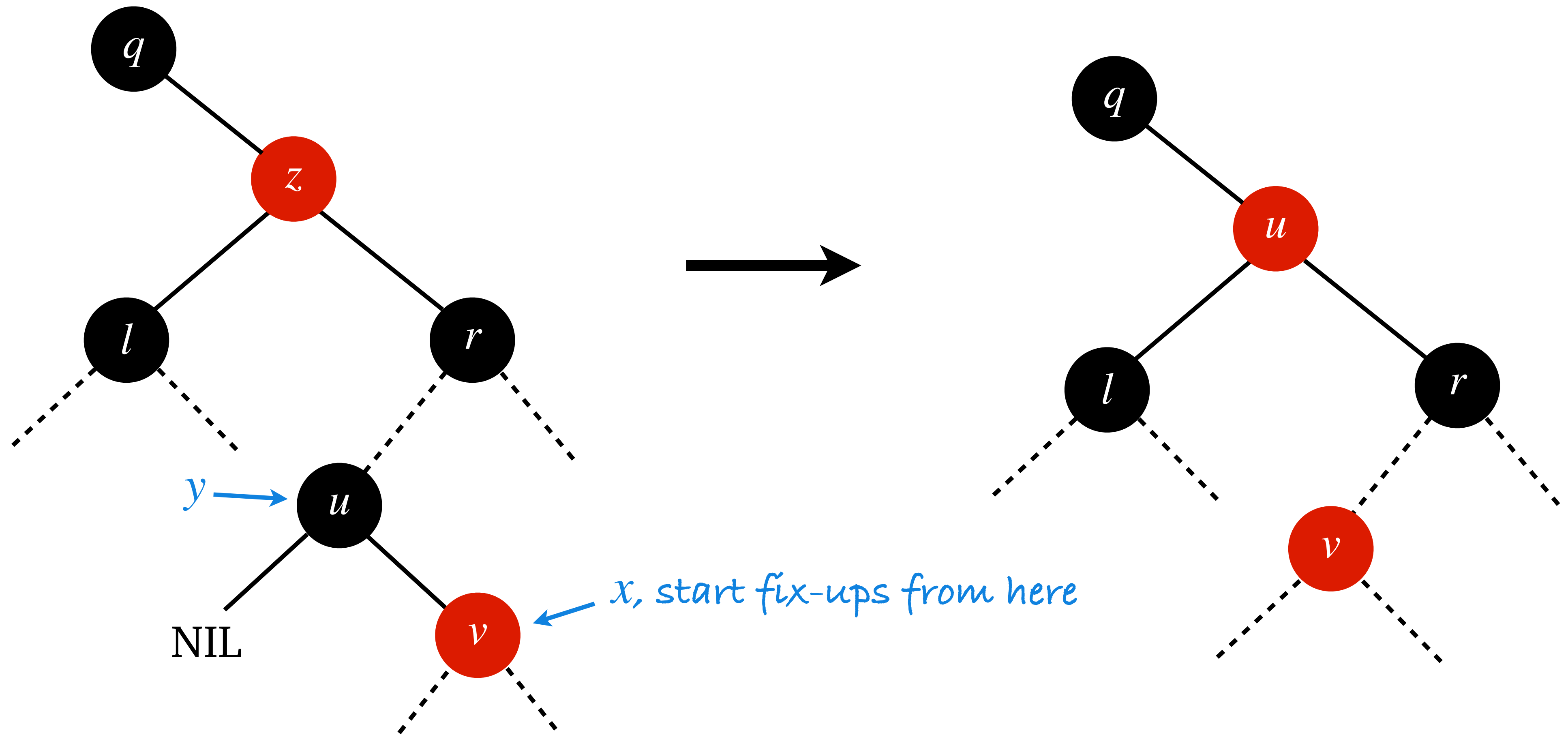
Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



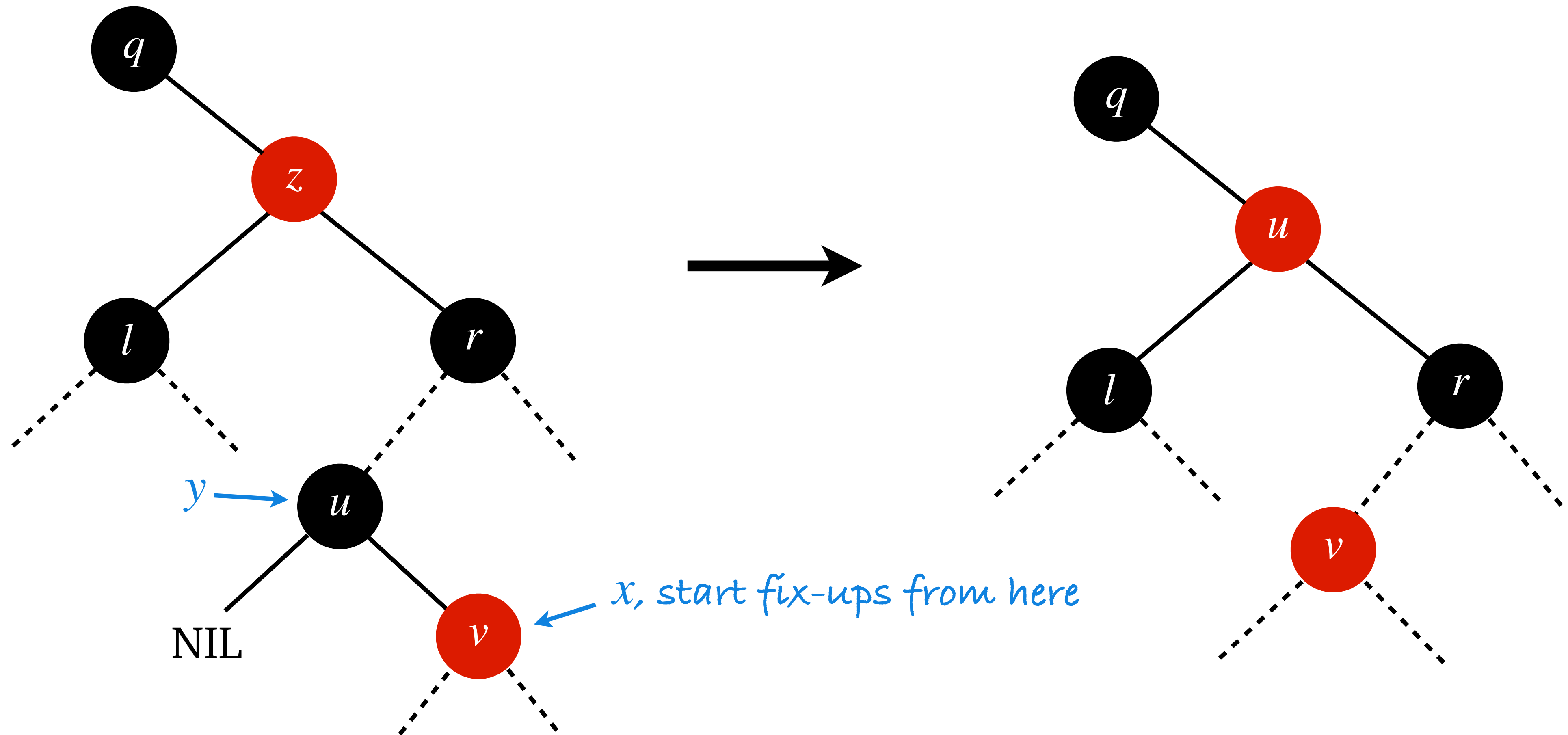
Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



Modifying a BST: Deletion

Case 3b: z has two (non-NIL) children where its right child has a left child.



Note: In this case, y is the successor of z and x is the only child of y .

RB-Trees: Deletion

RB-Trees: Deletion

Let z be the node to be deleted:

RB-Trees: Deletion

Let z be the node to be deleted:

- If z has at most one (non-NIL) child, then $y = z$ and x will be its only non-NIL child.

RB-Trees: Deletion

Let z be the node to be deleted:

- If z has at most one (non-NIL) child, then $y = z$ and x will be its only non-NIL child.
- Else, y will be the successor of z and x will be y 's only non-NIL child.

RB-Trees: Deletion

Let z be the node to be deleted:

- If z has at most one (non-NIL) child, then $y = z$ and x will be its only non-NIL child.
- Else, y will be the successor of z and x will be y 's only non-NIL child.

Note: x will be NIL if y has no non-NIL children.

RB-Trees: Deletion

Let z be the node to be deleted:

- If z has at most one (non-NIL) child, then $y = z$ and x will be its only non-NIL child.
- Else, y will be the successor of z and x will be y 's only non-NIL child.

Note: x will be NIL if y has no non-NIL children.

When do we need a fix-up?

RB-Trees: Deletion

Let z be the node to be deleted:

- If z has at most one (non-NIL) child, then $y = z$ and x will be its only non-NIL child.
- Else, y will be the successor of z and x will be y 's only non-NIL child.

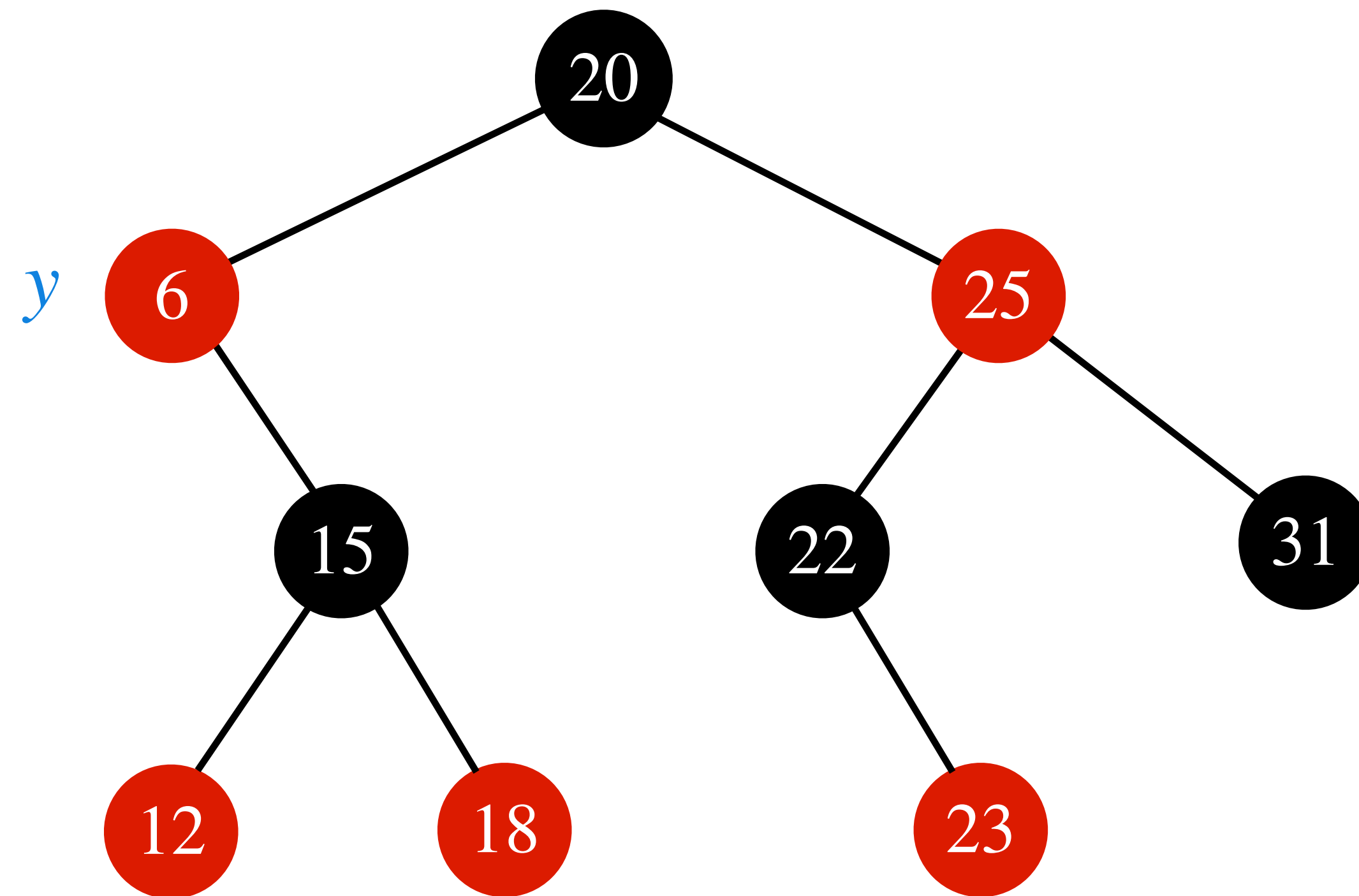
Note: x will be NIL if y has no non-NIL children.

When do we need a fix-up?

Only when y 's colour is black.

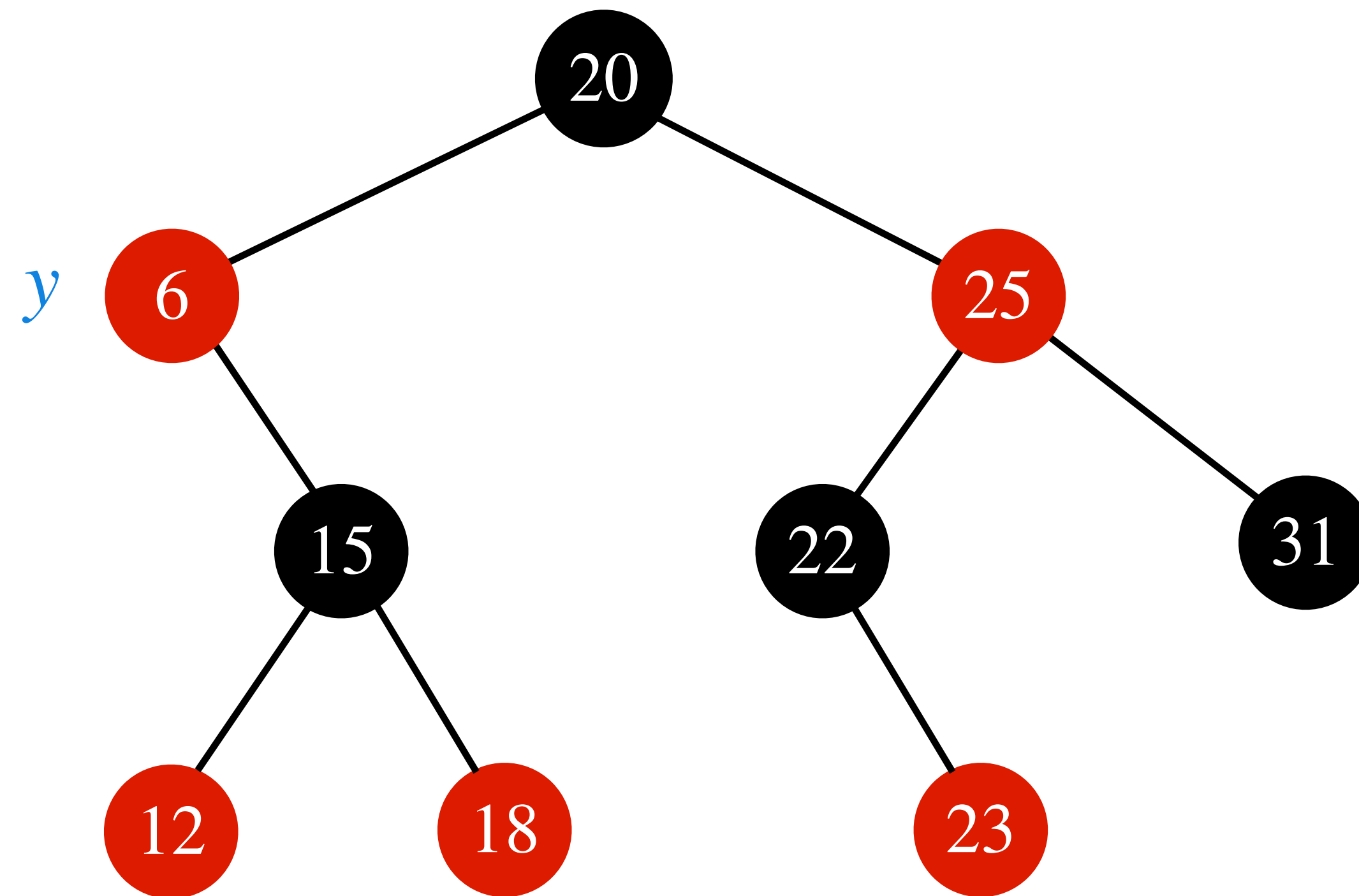
No fix-ups when y is Red

Property 1: Every node has a colour either **red** or black.



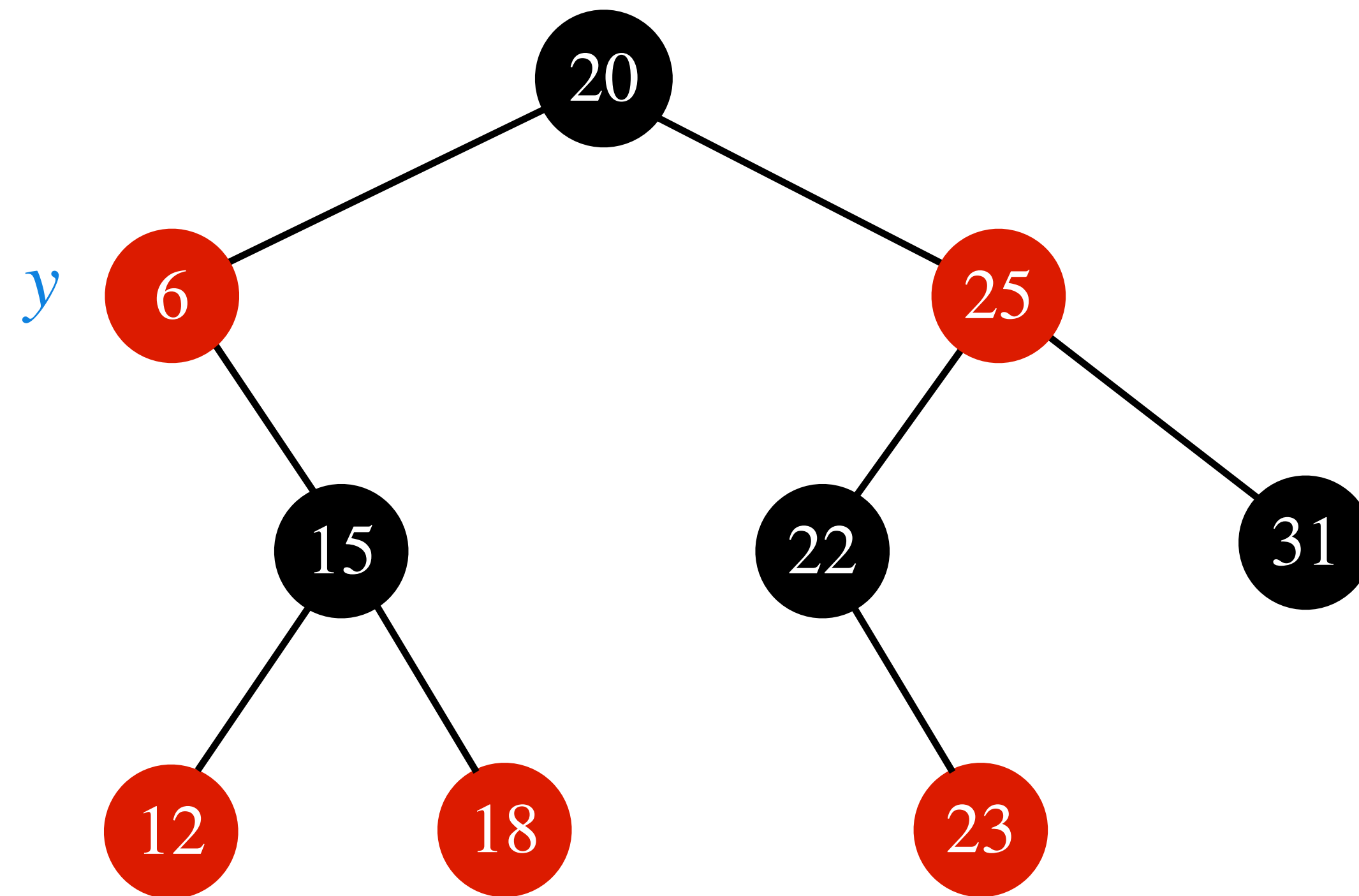
No fix-ups when y is Red

Property 2: Root is black.



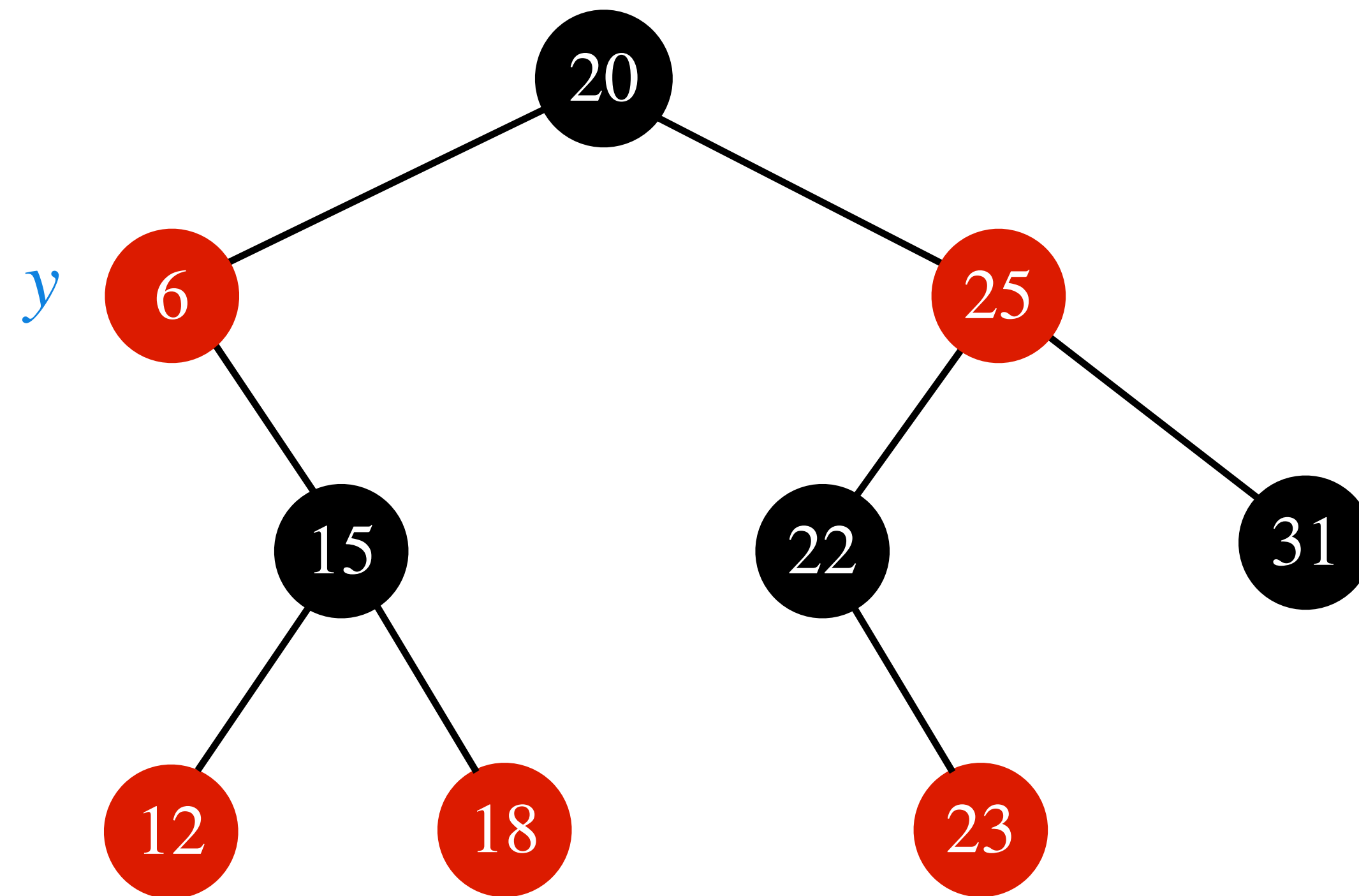
No fix-ups when y is Red

Property 3: Leaf nodes are NIL nodes which are black in colour.



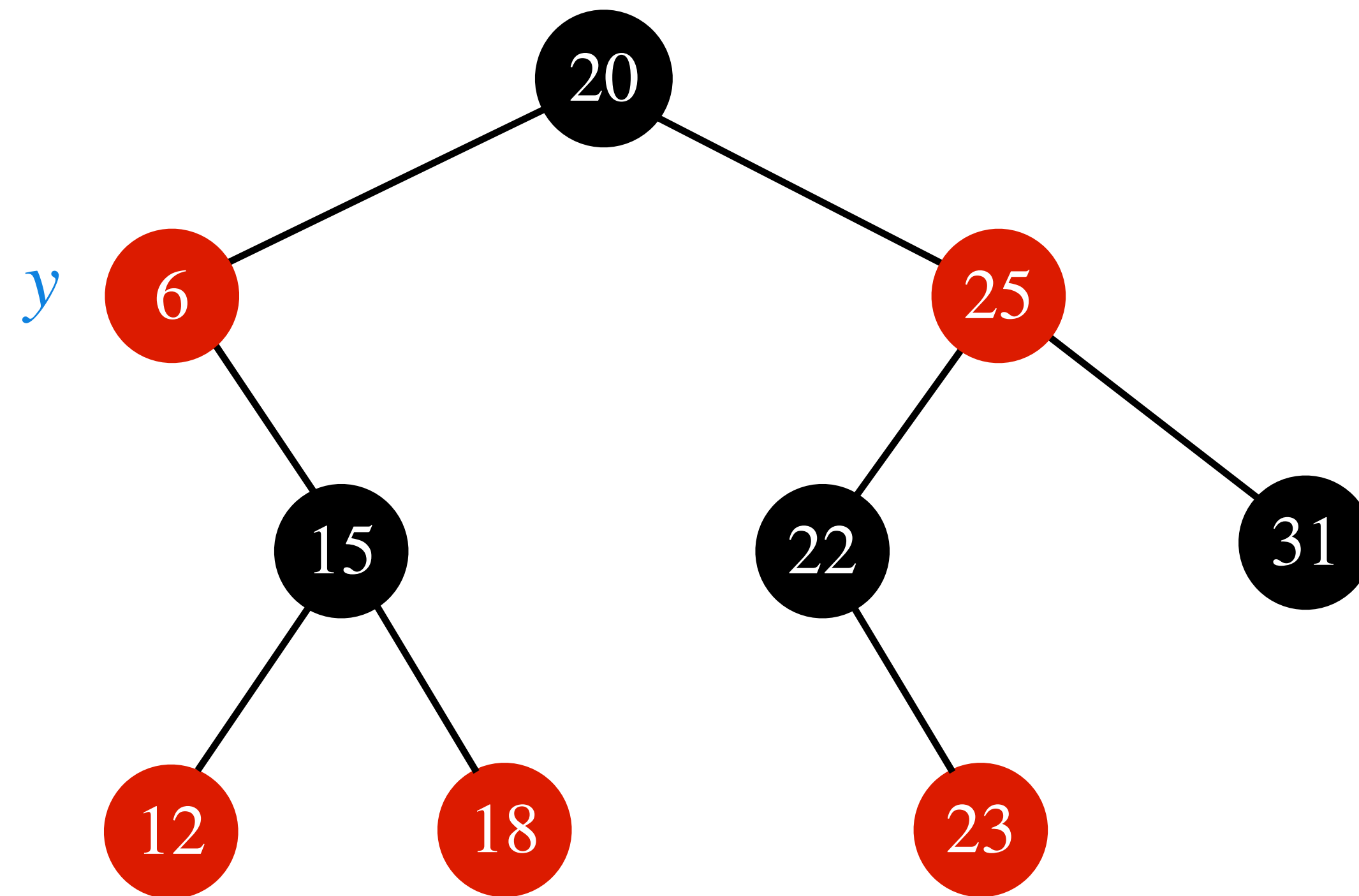
No fix-ups when y is Red

Property 4: Every internal (non-leaf) node has two children.



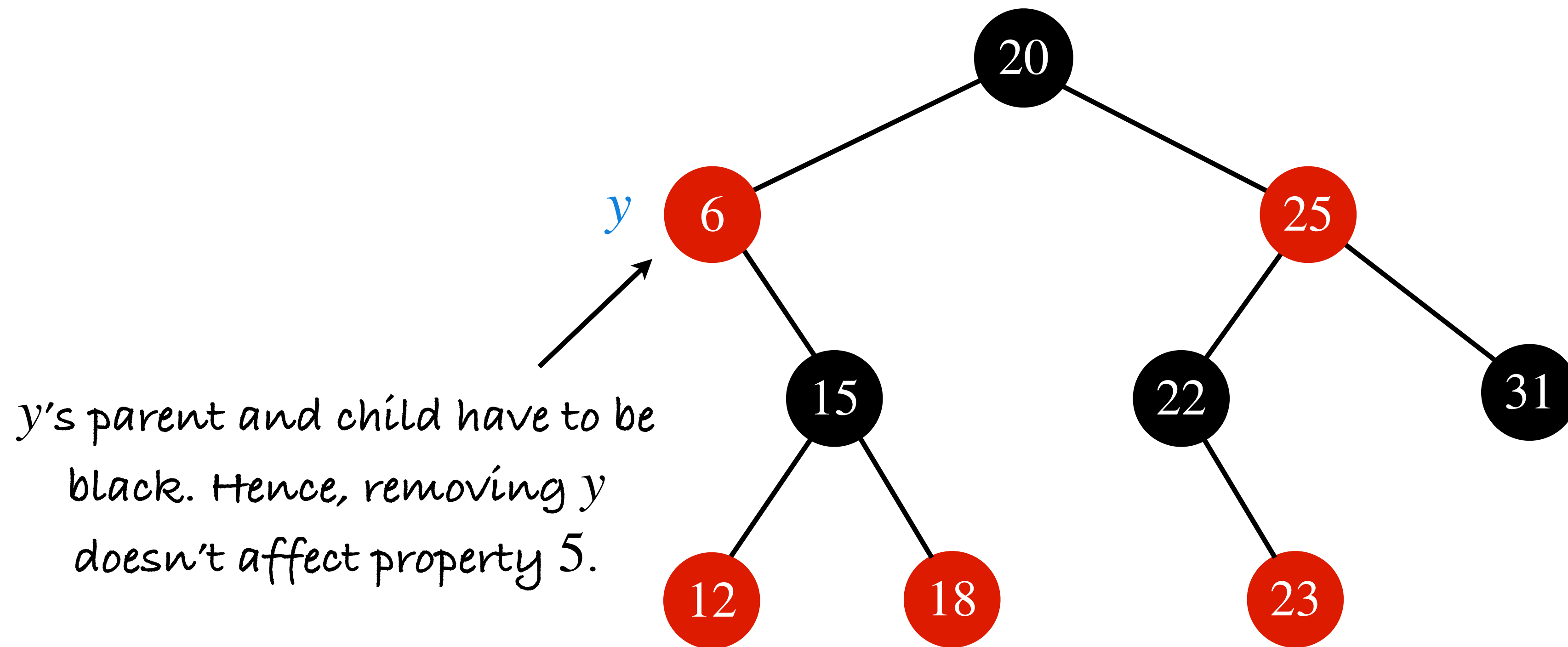
No fix-ups when y is Red

Property 5: If a node is **red**, then both its children are black.



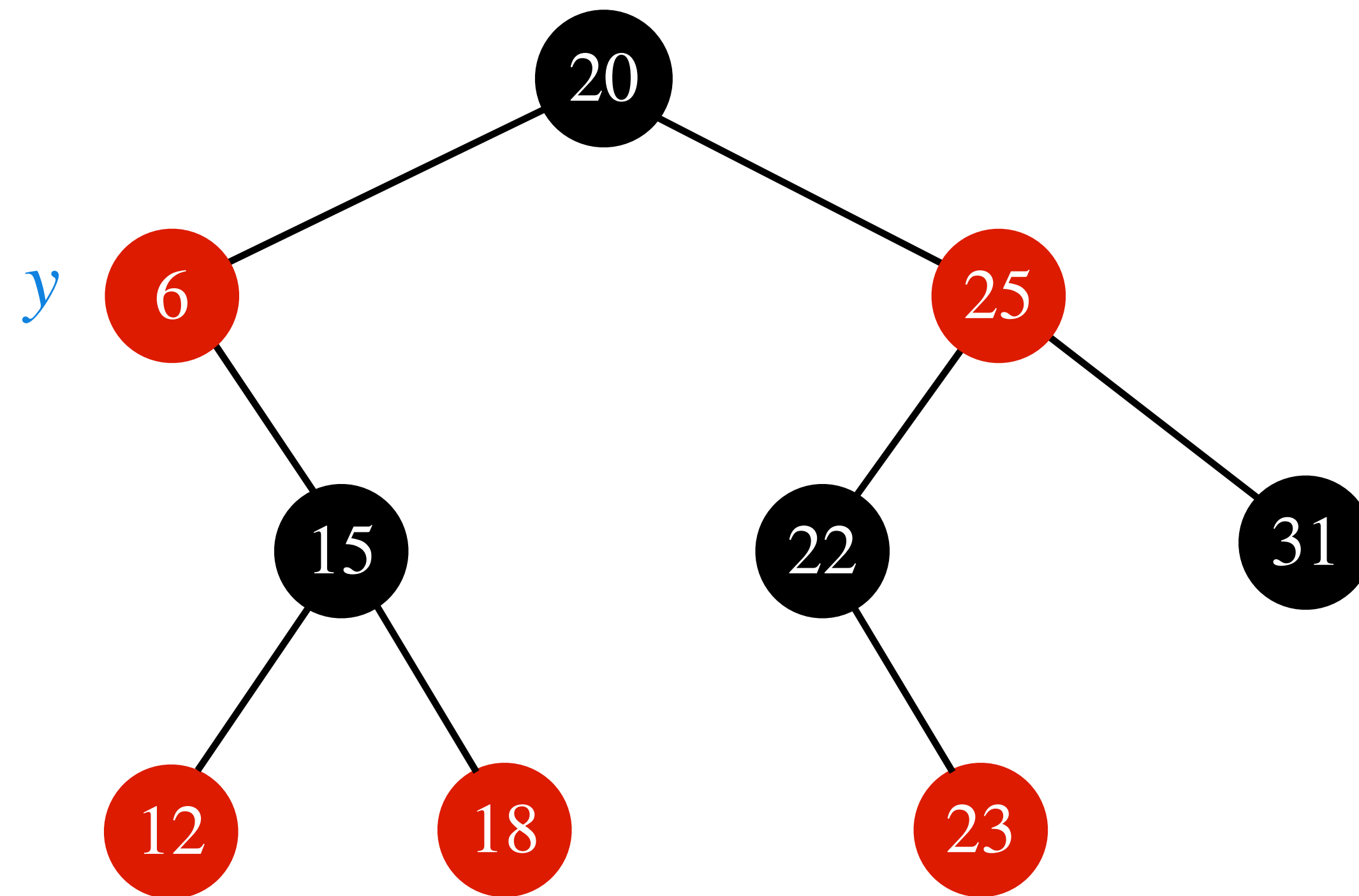
No fix-ups when y is Red

Property 5: If a node is **red**, then both its children are black.



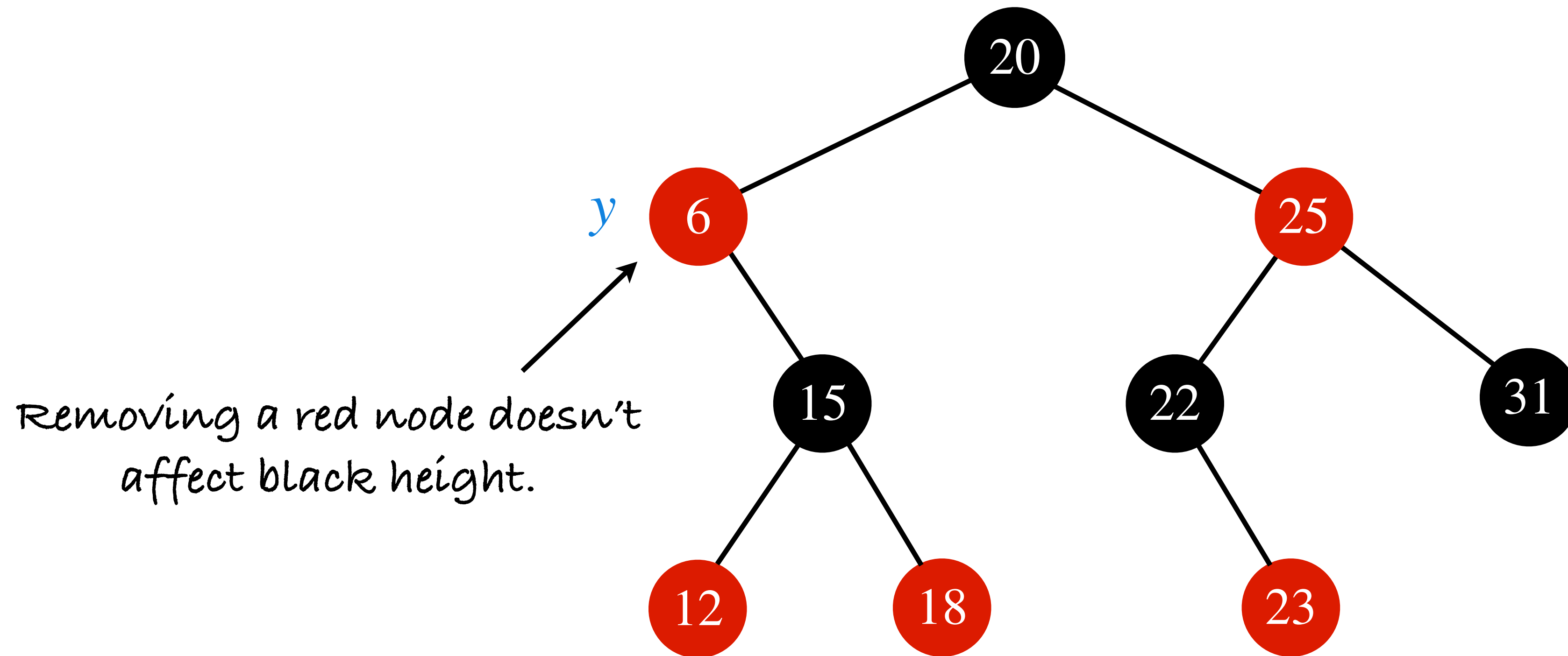
No fix-ups when y is Red

Property 6: Black-height consistency...



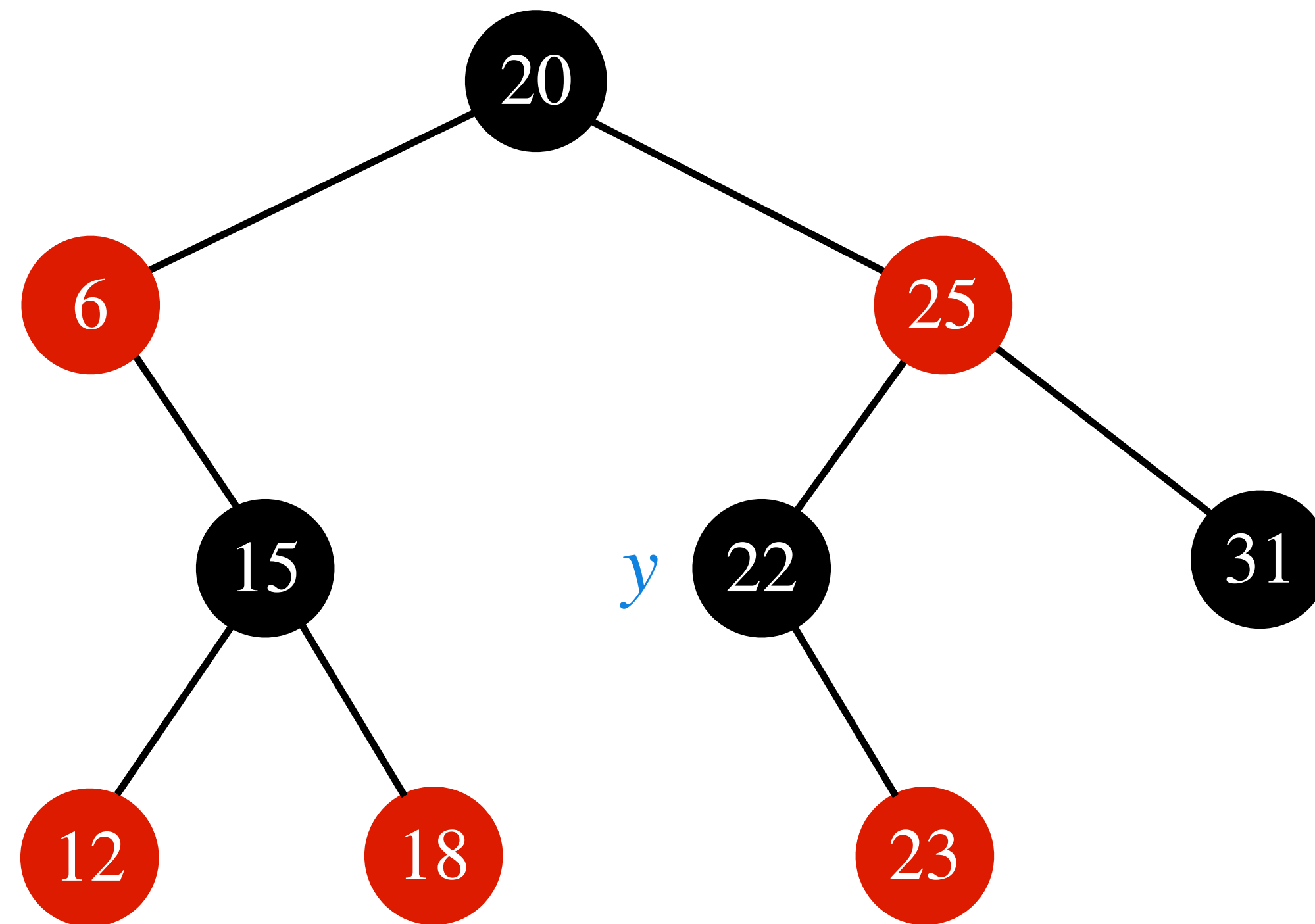
No fix-ups when y is Red

Property 6: Black-height consistency...



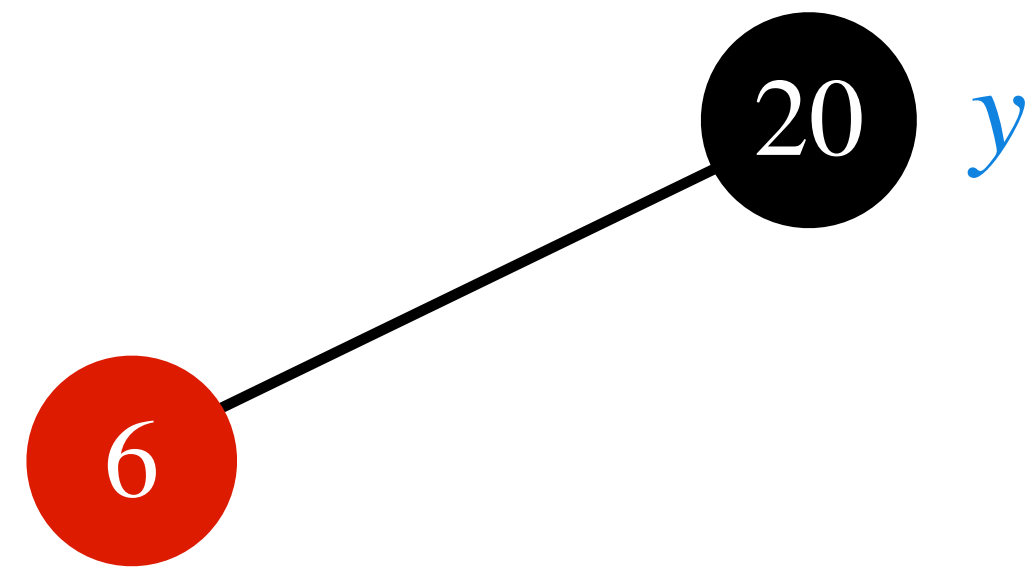
Fix-ups required when y is Black

Property 1: Every node has a colour either **red** or black.



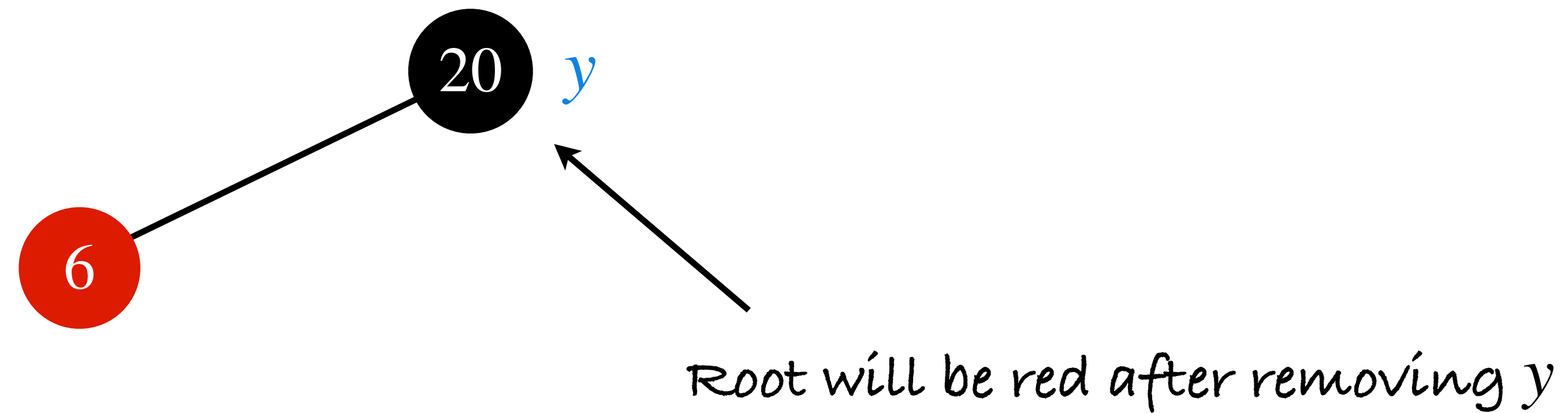
Fix-ups required when y is Black

Property 2: Root is black.



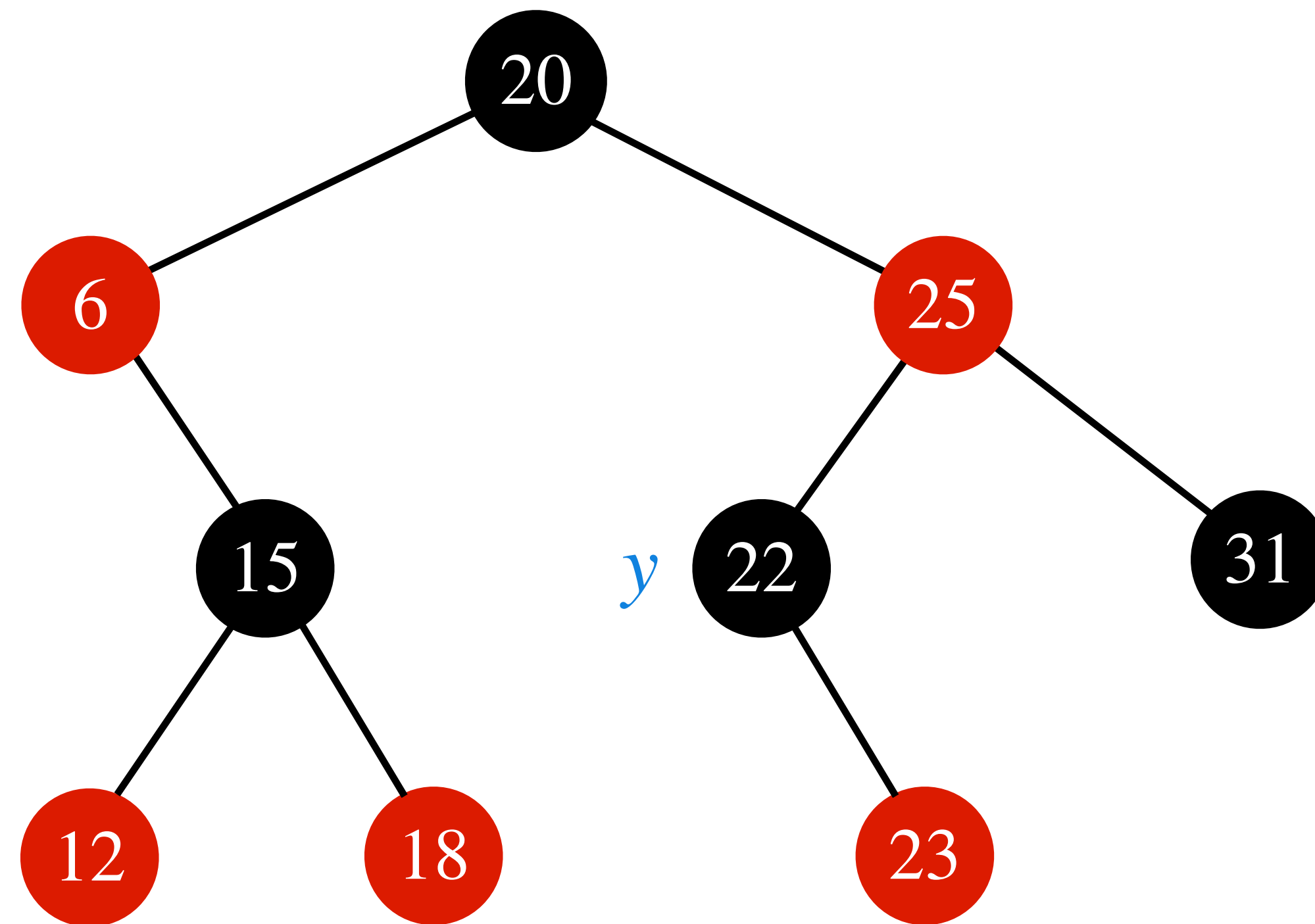
Fix-ups required when y is Black

Property 2: Root is black.



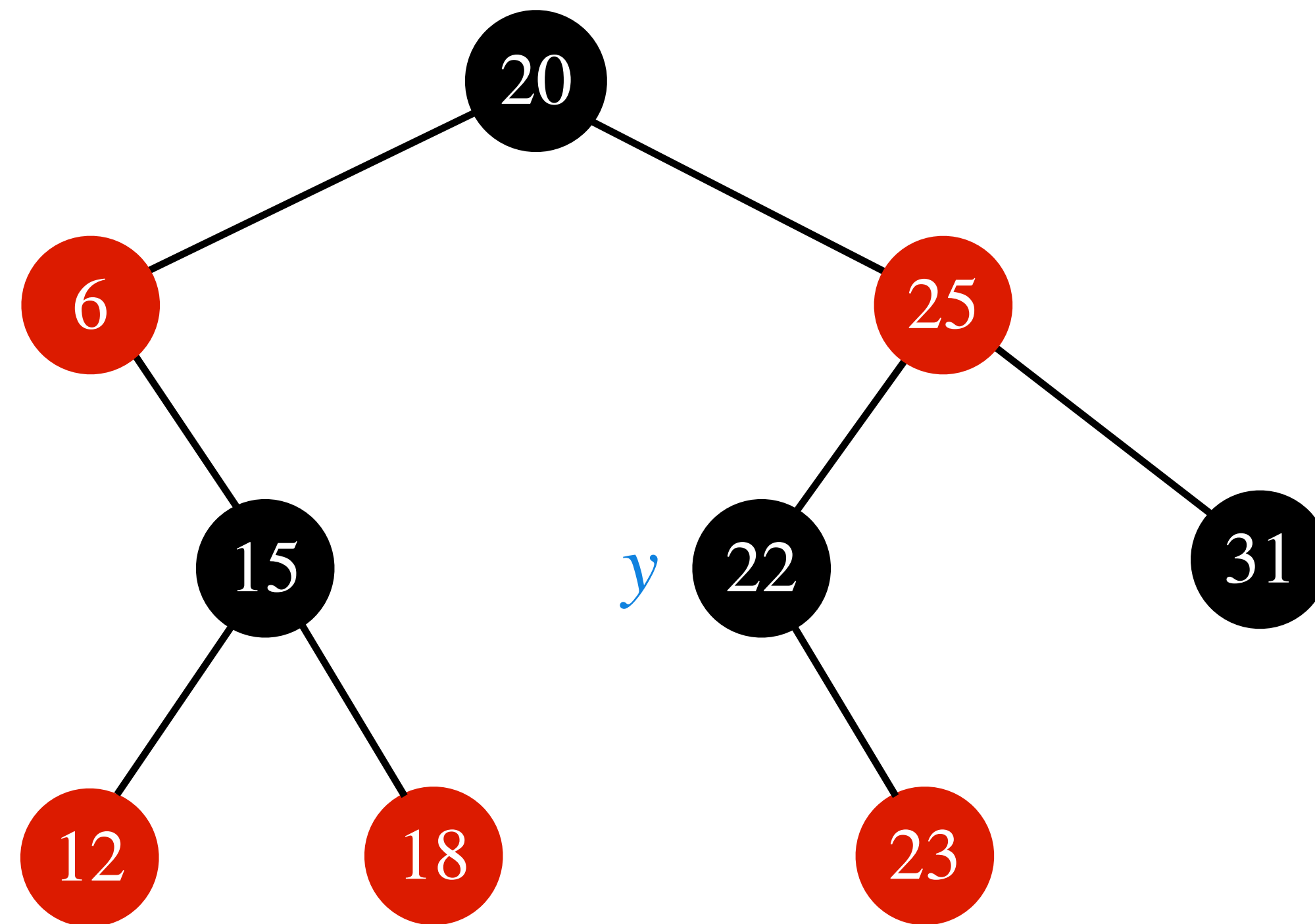
Fix-ups required when y is Black

Property 3: Leaf nodes are NIL nodes which are black in colour.



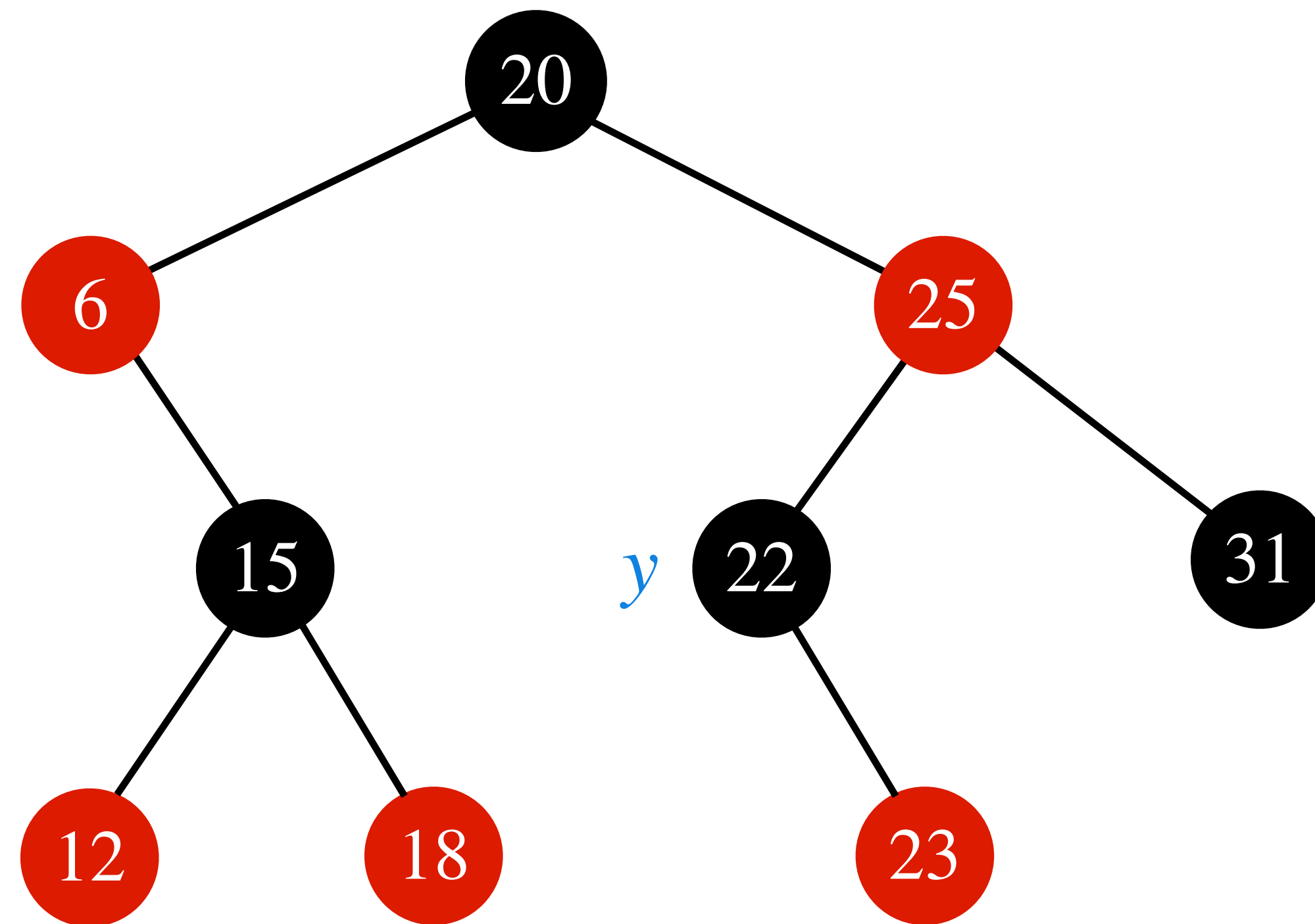
Fix-ups required when y is Black

Property 4: Every internal (non-leaf) node has two children.



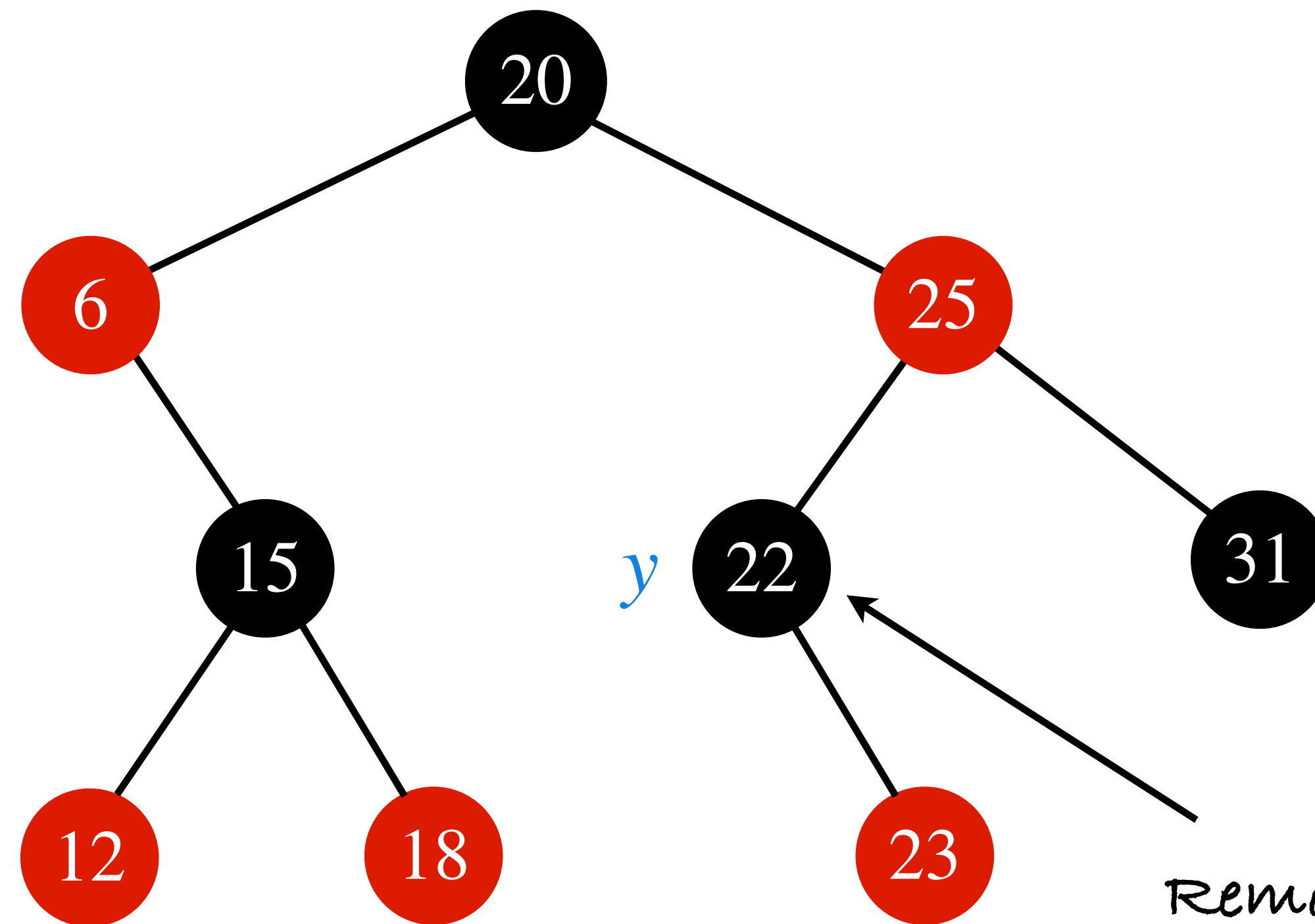
Fix-ups required when y is Black

Property 5: If a node is **red**, then both its children are black.



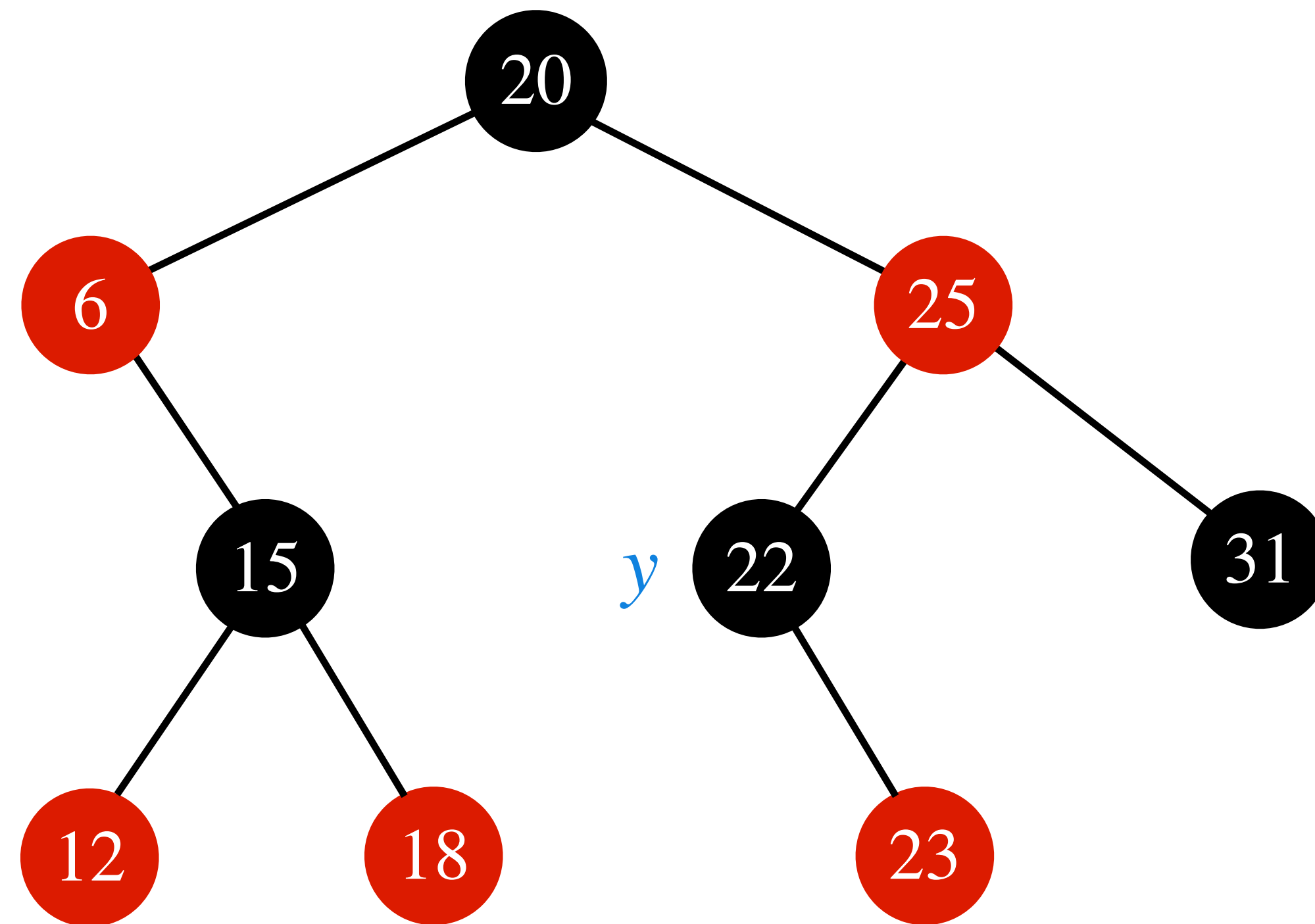
Fix-ups required when y is Black

Property 5: If a node is **red**, then both its children are black.



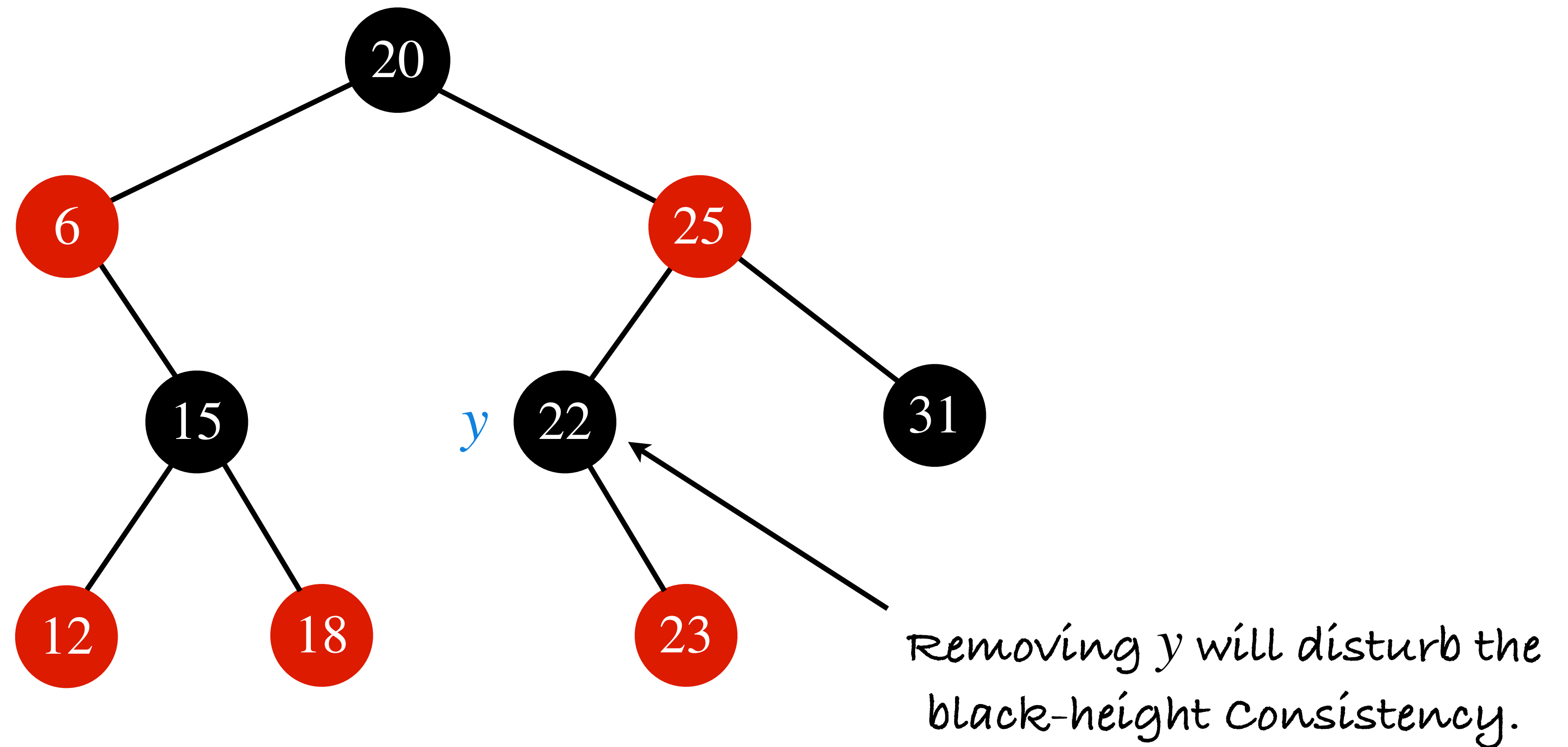
Fix-ups required when y is Black

Property 6: Black-height consistency...



Fix-ups required when y is Black

Property 6: Black-height consistency...



Fix-ups required when y is Black

Fix-ups required when y is Black

If y is black, then deleting y can violate the following properties:

Fix-ups required when y is Black

If y is black, then deleting y can violate the following properties:

- **Property 2:** Root is black.

Fix-ups required when y is Black

If y is black, then deleting y can violate the following properties:

- **Property 2:** Root is black.
- **Property 5:** If a node is red, then both its children are black.

Fix-ups required when y is Black

If y is black, then deleting y can violate the following properties:

- **Property 2:** Root is black.
- **Property 5:** If a node is red, then both its children are black.
- **Property 6:** Black-height consistency...

Fix-ups required when y is Black

If y is black, then deleting y can violate the following properties:

- **Property 2:** Root is black.
- **Property 5:** If a node is red, then both its children are black.
- **Property 6:** Black-height consistency...

Recall that x is y 's only non-NIL child.

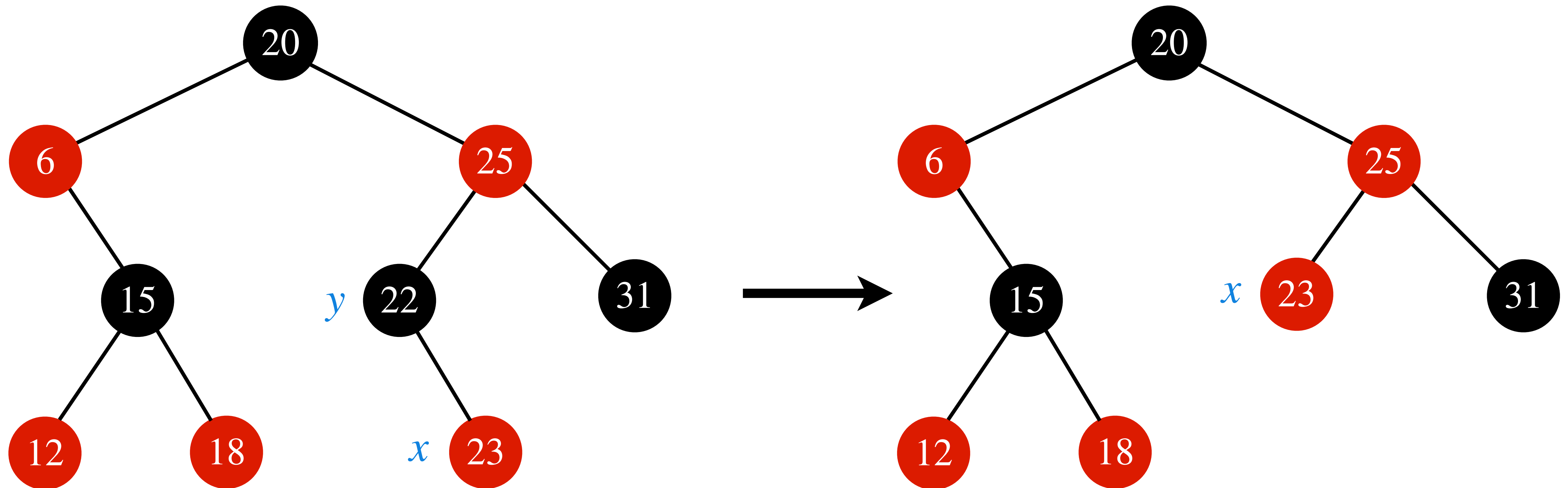
Fix-ups required when y is Black

If y is black, then deleting y can violate the following properties:

- **Property 2:** Root is black.
- **Property 5:** If a node is red, then both its children are black.
- **Property 6:** Black-height consistency...

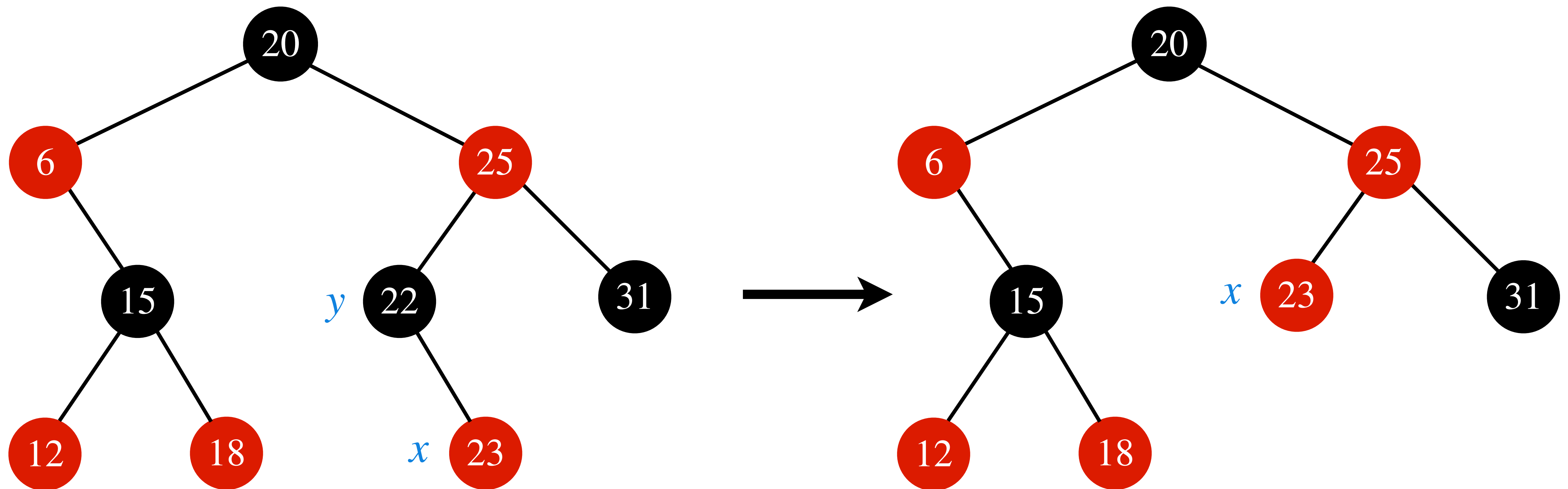
Recall that x is y 's only non-NIL child. (x will be NIL if y has no non-NIL children.)

Fix-up Idea



Fix-up Idea

Idea: Assume x has an extra black colour it wants to lose without violating RB-tree properties.



Skeleton for Fix-up

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. while $x \neq T.root$ and $x.colour = \text{BLACK}$

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

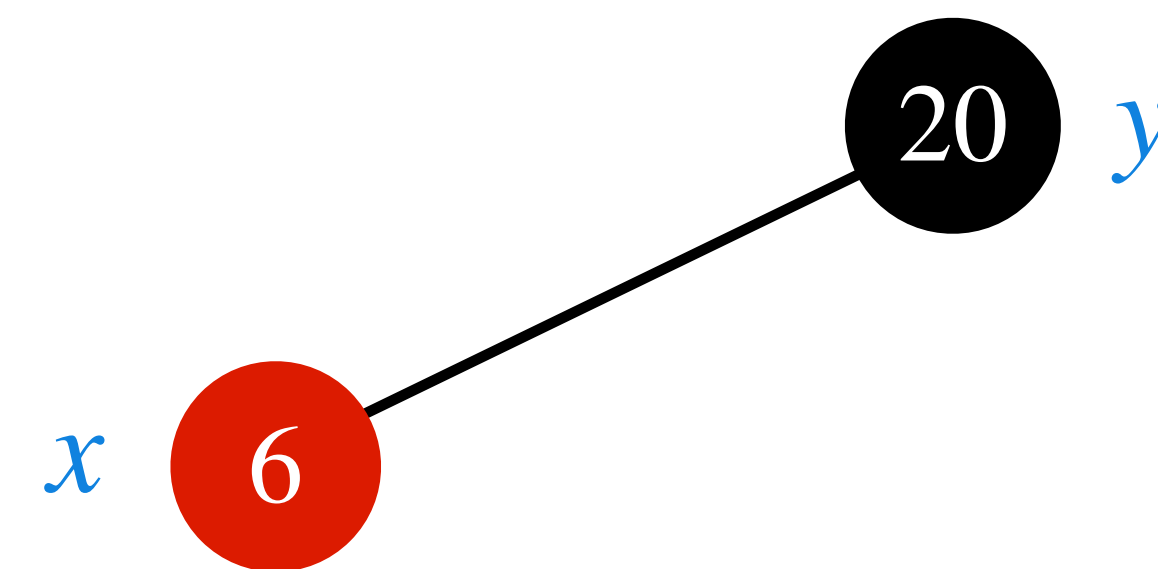
Property 2's violation will get fixed instantly.

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

Property 2's violation will get fixed instantly.



Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

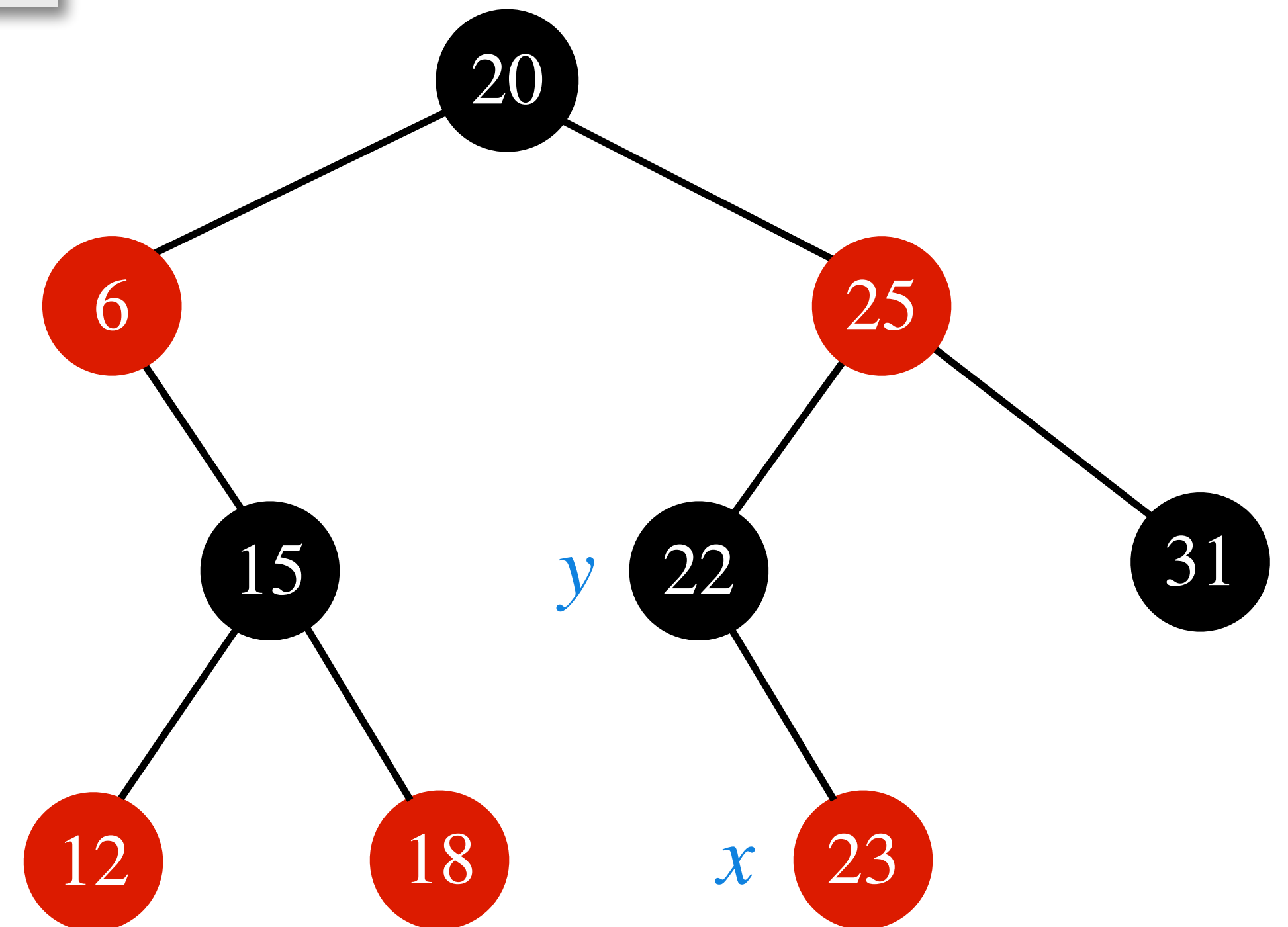
Property 5's violation will get fixed instantly.

Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

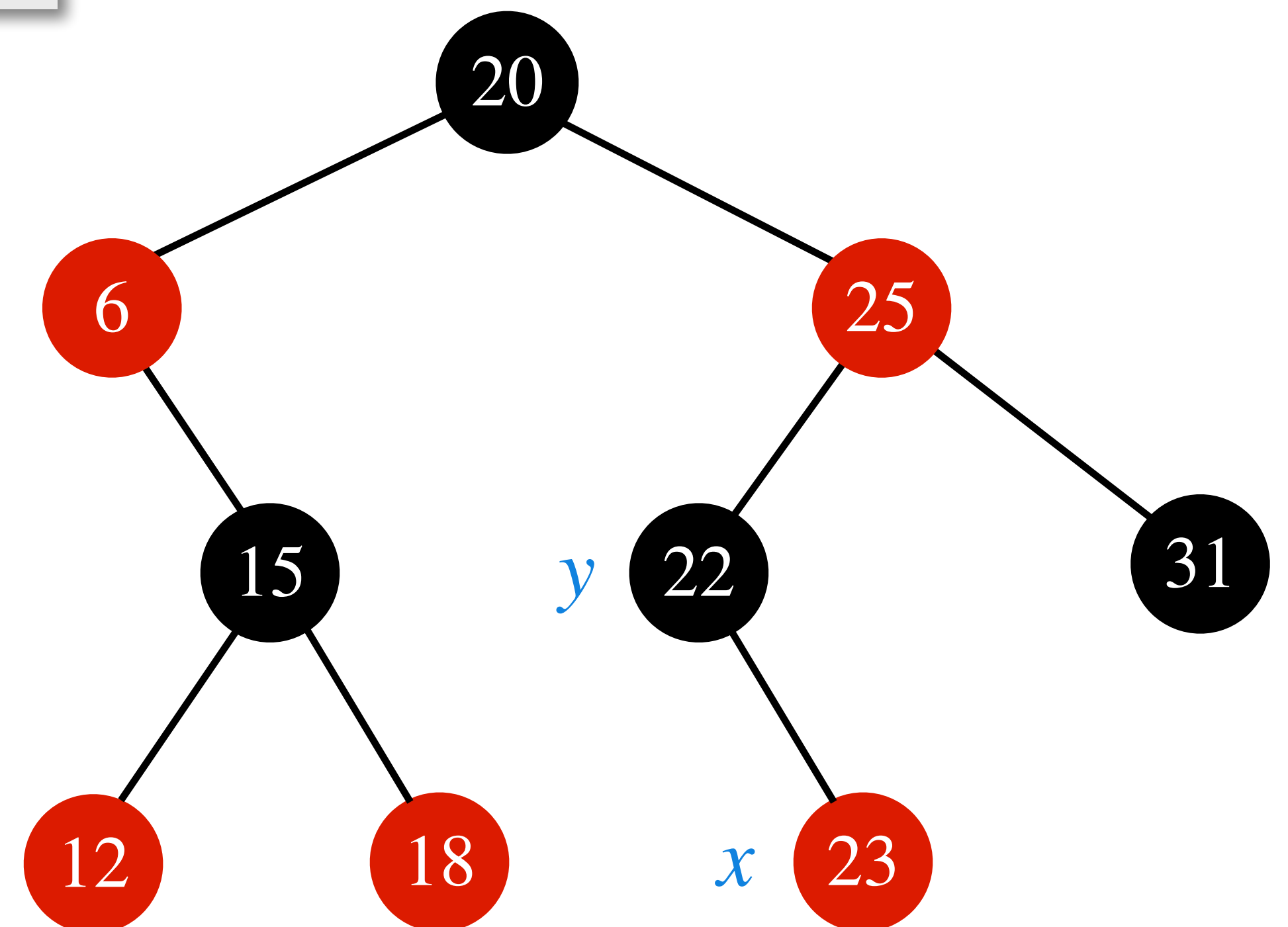
Property 5's violation will get fixed instantly.



Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

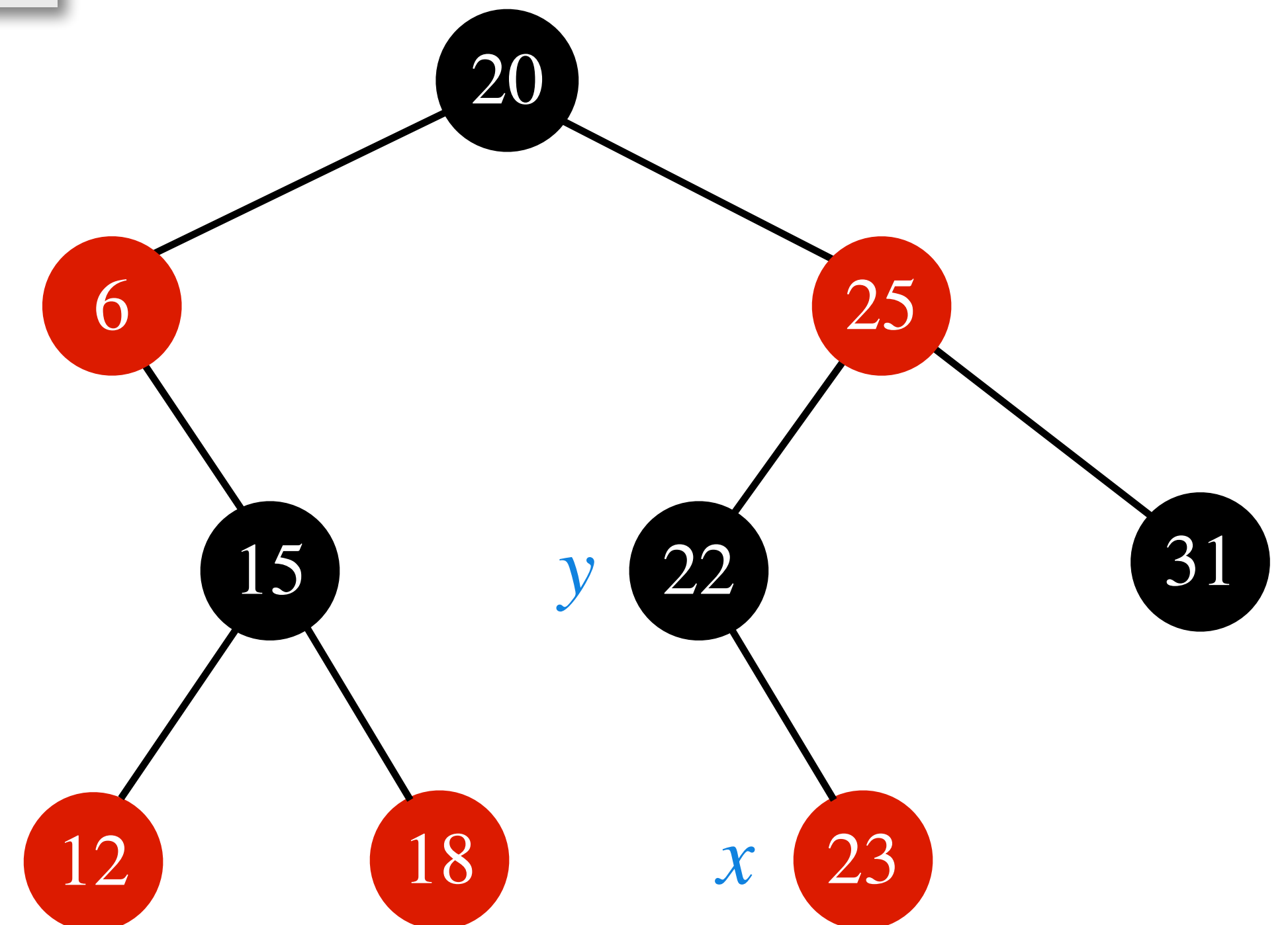


Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

Code will enter the loop only when **Property 6**

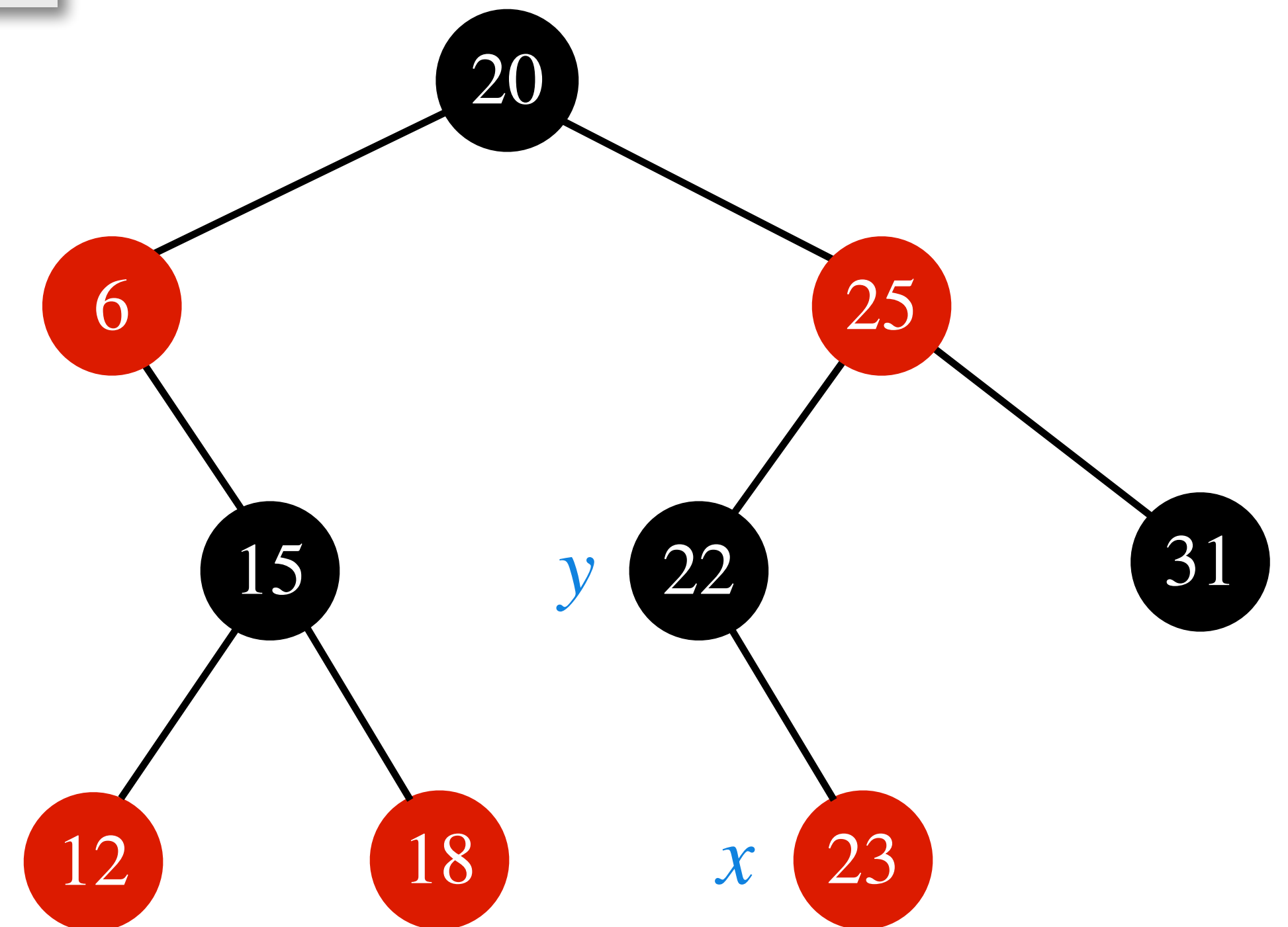


Skeleton for Fix-up

RB-Delete-Fixup(T, x):

1. **while** $x \neq T.root$ and $x.colour = \text{BLACK}$
2. several cases handled by rotations and recolouring
3. $x.colour = \text{BLACK}$

Code will enter the loop only when **Property 6** is the only property that is getting violated.



Cases for Deletion

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x 's sibling w is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x 's sibling w is red.
- **Case 2:** x 's sibling w is black, and both of w 's children are black.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x 's sibling w is red.
- **Case 2:** x 's sibling w is black, and both of w 's children are black.
- **Case 3:** x 's sibling w is black, and w 's left child is red, and w 's right child is black.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x 's sibling w is red.
- **Case 2:** x 's sibling w is black, and both of w 's children are black.
- **Case 3:** x 's sibling w is black, and w 's left child is red, and w 's right child is black.
- **Case 4:** x 's sibling w is black, and w 's right child is red.

Cases for Deletion

There are four cases when deletion causes only **Property 6's** violation:

- **Case 1:** x 's sibling w is red.
- **Case 2:** x 's sibling w is black, and both of w 's children are black.
- **Case 3:** x 's sibling w is black, and w 's left child is red, and w 's right child is black.
- **Case 4:** x 's sibling w is black, and w 's right child is red.

Idea: Assume x has an extra black colour it wants to lose without violating RB-tree properties.